Nondeterministic Linear Time Is Strictly Contained in Deterministic Linear Space

Charlie Cruz*

November 1, 2026

Abstract

Paul, Pippenger, Szemerédi and Trotter proved in 1983 that, for multitape Turing machines, nondeterministic linear time is more powerful than deterministic linear time. We prove that it is less powerful than deterministic linear space: $\mathrm{NTIME}(n) \subsetneq \mathrm{DSPACE}(n)$. More precisely, some language in $\mathrm{DSPACE}(n)$ requires nondeterministic time $\Omega\big(n\sqrt{\log n}/(\log\log n)^2\big)$ at every large input length. The proof simulates nondeterministic time t in deterministic space $O\big(t(\log\log t)^2/\sqrt{\log t}\big)$ and applies the space hierarchy theorem. To our knowledge this is the first simulation of nondeterministic time in space $o(t)$, deterministic or nondeterministic. Known simulations of time in small space recompute parts of the computation. For a nondeterministic machine, a recomputed part would have to repeat guesses that were never stored, so our simulation counts the accepting computations instead. To count them, we fix the moves of the work heads between blocks of cells. The computations of length t that make these moves then correspond to the satisfying assignments of a system of local constraints of logarithmic arity on $O(t)$ Boolean variables, each variable in at most two constraints. We show that the counting algorithm behind the recent refutations of the Exponential Time Hypothesis, in Kintali's formulation, counts the satisfying assignments of such a system with N variables in space $O\big(N(\log\log N)^2/\sqrt{\log N}\big)$, and we reprove the construction it uses, so that the paper is self-contained. We also obtain $\mathrm{NTIME}\big(t\sqrt{\log t}/(\log\log t)^2\big) \subseteq \mathrm{DSPACE}(t)$, partial progress on a question of Hopcroft, Paul and Valiant. The author will be working in the coming weeks to rewrite the argument and strengthen the claims of this draft.

1 Introduction

For multitape Turing machines, linear time and linear space are related by the chain of inclusions

$$\mathrm{DTIME}(n) \subseteq \mathrm{NTIME}(n) \subseteq \mathrm{DSPACE}(n) \subseteq \mathrm{NSPACE}(n).$$

Paul, Pippenger, Szemerédi and Trotter proved in 1983 that the first inclusion is strict [14]. We prove that the second one is strict as well:

$$\mathrm{NTIME}(n) \subsetneq \mathrm{DSPACE}(n).$$

Since $\mathrm{DSPACE}(n)$ and $\mathrm{NSPACE}(n)$ are the classes of deterministic and of all context-sensitive languages [12], this means that some deterministic context-sensitive language cannot be accepted by any nondeterministic multitape machine in linear time.

*Draft prepared through conversations with LLMs. Section 1.3 explains what the results depend on, and the Acknowledgments explain how the paper was prepared.

The usual way to prove such a separation is to simulate time t in space $o(t)$ and then apply the space hierarchy theorem. For deterministic machines this has been possible since 1975. Hopcroft, Paul and Valiant [9] simulated deterministic time t in space $O(t/\log t)$, which separates DTIME(n) from DSPACE(n), and Williams [18], building on the tree evaluation algorithm of Cook and Mertz [5], recently improved the space to $O(\sqrt{t \log t})$. Both simulations save space by recomputation: they keep a few intermediate results and recompute the others when they are needed. Hopcroft, Paul and Valiant pointed out that recomputation breaks down for nondeterministic machines. A recomputed part of a nondeterministic computation has to repeat the guesses of the original, and "we cannot be sure that the same sequence of choices is made the second time" [8, §2], while storing the guesses costs as much space as the computation itself. They left open whether NTIME($t \log t$) $\subseteq$ NSPACE(t). Santhanam [16] related the question whether NTIME(t) = NSPACE(t) to separators of certain graphs, but no simulation of nondeterministic time t in space $o(t)$, deterministic or nondeterministic, was known.

Nor do translational arguments settle the question. Padding arguments in the style of Book [2] show that P and NP both differ from DSPACE(n), without deciding which inclusions fail, because P and NP are closed under polynomial-time reductions and DSPACE(n) is not. The closure properties that these arguments use are the same for NTIME(n) and DSPACE(n): both classes are closed under reductions computable in linear time with output of linear length, and neither is closed under polynomial-time reductions. The equality NTIME(n) = DSPACE(n) would translate upward to NTIME(T) = DSPACE(T) for time-constructible $T(n) \geq n$, and, since alternating time is contained in deterministic space of the same order [3], it would collapse the linear-time alternating hierarchy to NTIME(n) and make NTIME(n) closed under complement. To our knowledge, none of these consequences was known to be false, and no proof of NTIME(n) $\neq$ DSPACE(n) had appeared.

We avoid the obstacle that Hopcroft, Paul and Valiant pointed out by counting the accepting computations instead of following them, with an algebraic algorithm whose intermediate objects are much smaller than a computation. The algorithm comes from the recent refutations of the Exponential Time Hypothesis. OpenAI [13, Theorem 1.1] showed that the satisfying assignments of a k-CNF with N variables and a linear number of clauses can be counted in time $2^{O(N/\sqrt{\log N})}$, Anthropic [1] independently gave a $2^{o(n)}$-time algorithm for 3-SAT, and Kintali [11] gave a simplified, self-contained proof of the construction of [13]. As written, these algorithms use space polynomial in N, but Kintali's algorithm never needs an assignment to all N variables at once. After a deformation of the constraints, the variables split into groups that interact only through a few linear measurements, and the count becomes a sum, over the values of the measurements and over small sets of variables, of products of sums over single groups. We show that the whole algorithm can be run while storing little more than the indices of this sum, which take $o(N)$ bits, and that it applies to constraints of logarithmic arity as well as to clauses. A computation of length t, once the moves of its work heads between blocks of cells are fixed, is described by such a system of constraints on $O(t)$ variables. These moves can be written with $O(t/\log t)$ bits, so all of them can be tried in turn, and acceptance can be decided in space $o(t)$.

1.1 Results

Theorem 1 (Main Theorem)**.** NTIME(n) $\subsetneq$ DSPACE(n). *More precisely, let*

$$\rho(n) = \frac{n\sqrt{\log n}}{(\log \log n)^2}.$$

There is a language $L \in \mathrm{DSPACE}(n)$ such that every nondeterministic multitape machine that decides L, or its complement, runs for at least $c\,\rho(n)$ steps on some input of length n, for some $c > 0$ and all large n. In particular L is in neither $\mathrm{NTIME}(T)$ nor $\mathrm{coNTIME}(T)$ for any $T = o(\rho)$.

Theorem 1 follows from the next theorem and the space hierarchy theorem. Write

$$\kappa(N) = \frac{N(\log\log N)^2}{\sqrt{\log N}},$$

with harmless adjustments for small arguments, so that $\kappa(N) = o(N)$.

Theorem 2 (Simulation)**.** *For every function $T(n) \geq n$,*

$$\mathrm{NTIME}(T) \cup \mathrm{coNTIME}(T) \subseteq \mathrm{DSPACE}\big(\kappa(T)\big).$$

The simulating machine runs in time $2^{O(\kappa(T))}$.

The simulation rests on the following theorem, the main technical result of the paper. Constraint systems, and what it means for one to be given in logarithmic space, are defined in Section 2.

Theorem 3 (Counting in sublinear space)**.** *Let c and c_0 be constants, and consider constraint systems on N Boolean variables that are given in logarithmic space by a fixed transducer (Section 2), in which every variable lies in at most three scopes and every scope has at most $c\log_2 N$ variables. Given the input of the transducer and a prime p with $4N^2 < p \leq N^{c_0}$, the number of satisfying assignments modulo p can be computed in deterministic space $O(\kappa(N))$, with the constant depending on c, c_0 and the transducer. In particular, the satisfiability of a 3-CNF with N variables and $O(N)$ clauses, in which every variable occurs in at most three clauses, can be decided in deterministic space $O(\kappa(N))$.*

The same algorithm computes sums of products of integer-valued tables (Remark 4.7).

Theorem 2 has further consequences, which Section 6 proves. The separation holds at every space bound: for every space-constructible $S(n) \geq n$,

$$\mathrm{NTIME}(S) \subsetneq \mathrm{DSPACE}(S) \quad \text{and} \quad \mathrm{NTIME}(S) \subsetneq \mathrm{NSPACE}(S),$$

and $\mathrm{NTIME}(n)$ is even strictly contained in $\mathrm{DSPACE}\big(n/(\log n)^{1/2-\varepsilon}\big)$ for every rational $\varepsilon > 0$ (Theorem 6.2). For every $t(n) \geq n$ we obtain $\mathrm{NTIME}\big(t\sqrt{\log t}/(\log\log t)^2\big) \subseteq \mathrm{DSPACE}(t)$ (Corollary 6.3). Hopcroft, Paul and Valiant asked for the factor $\log t$, in nondeterministic space; we obtain the factor $\sqrt{\log t}$, up to $\log\log$ factors, in deterministic space, and their question remains open. Since the simulation counts, it applies equally to probabilistic time with bounded or unbounded error and to parity time, and the language of Theorem 1 is in none of these classes for time $o(\rho)$ (Corollary 6.5). For deterministic time, Williams's simulation already gives much more: the language of Theorem 1 is not in $\mathrm{DTIME}(o(n^2/\log n))$.

1.2 Proof overview

The proof has three steps:

$\mathrm{NTIME}(t)$	$\Longrightarrow$	counting the satisfying assignments of $2^{O(t/\log t)}$ constraint systems, each with $O(t)$ variables, logarithmic arity and bounded occurrence	Section 5
	$\Longrightarrow$	deterministic space $O(\kappa(t)) = o(t)$, by counting in sublinear space	Sections 3, 4
	$\Longrightarrow$	$\mathrm{NTIME}(n) \subsetneq \mathrm{DSPACE}(n)$, by the space hierarchy theorem	Section 6

Step 1: computations as constraint systems (Section 5). Following the block decomposition of Hopcroft, Paul and Valiant [9], we cut a computation of length t into intervals of $b \approx \log t$ steps and every work tape into blocks of b cells. During one interval a head moves at most b cells, so it ends the interval in its block or in a neighboring one, and a *head schedule* records these moves between blocks, for every interval and every work tape. A head schedule has $O(t/\log t)$ bits, so all of them can be enumerated within our space bound. Once the head schedule is fixed, each interval works on blocks that are known in advance. It reads its b choice bits and the $O(b)$ bits of data with which it enters, namely the state, the head positions and the contents of the blocks around the work heads, and these determine the data with which it leaves. Each interval therefore becomes one constraint of logarithmic arity, and each piece of data that an interval leaves behind is read by at most one later interval (Figure 1). The result is a constraint system with $O(t)$ Boolean variables, each in at most two constraints, whose satisfying assignments correspond one-to-one to the choice strings whose computation follows the head schedule and accepts. Summed over all head schedules, these counts give the number of choice strings that lead to acceptance, which is positive if and only if the machine accepts.

Step 2: counting in sublinear space (Sections 3 and 4). It remains to count the satisfying assignments of a bounded-occurrence constraint system with $N = O(t)$ variables in space $o(N)$, which rules out exhaustive search: the current assignment alone takes N bits. Kintali's algorithm replaces the sum over $\{0,1\}^N$ by sums over much smaller objects. It deforms the inputs of the constraints by polynomials in a new variable T, so that the count is $H(0)$ for a polynomial H, and so that at each of Q nodes $T = 1, \ldots, Q$ the variables split into r groups that interact only through integer linear measurements, each reading a single group, whose values take $o(N/r)$ bits in all. Fixing the values of the measurements decouples the groups, and the sum over all assignments becomes a sum, over the values of the measurements, of products of sums over single groups of $O(N/r)$ variables. The values of H at the nodes do not determine $H(0)$, because the degree of H is much larger than Q, but the Taylor expansions of H at the nodes do, and they have the same decoupled form, with an extra sum over small sets of variables called supports. A constraint enters the construction only through the multilinear extension of its table, so the constraints need not be clauses.

Implemented directly, the algorithm stores objects of at least linear size: the group schedules, which assign the variables and the constraints to groups at all nodes, the coefficients of the deformed forms, the Taylor coefficients from which $H(0)$ is recovered by Hermite interpolation, and the supports. We avoid all of them. A pairwise independent hash function generates the group schedules from a seed of $O(\log^2 N)$ bits, every coefficient is computed from an explicit formula when it is needed, a finite-field identity recovers $H(0)$ from values of polynomials whose low-order coefficients are the Taylor data, and supports are written in a compact code. The count modulo a prime then becomes a single explicit formula (Proposition 4.3), whose summation indices are a node, a field element, a support, a tuple of measurement values and an assignment to one group, and whose terms can each be computed in polylogarithmic space. Evaluated by nested loops, the formula needs only the space to write down its indices. A support has $O(N/r)$ elements, written with $O(\log r)$ bits each, the measurement values take $o(N/r)$ bits, and a group has $O(N/r)$ variables, so the indices take $O(N \log r/r)$ bits, which is $O(\kappa(N))$ for the $r \approx \sqrt{\log N}/\log\log N$ groups that our choice of parameters allows.

Step 3: the space hierarchy (Section 6). Enumerating the head schedules and counting for each of them, with a doubling search for the running time, gives $\mathrm{NTIME}(t) \subseteq \mathrm{DSPACE}(\kappa(t))$ (Theorem 2). Since $\kappa(t) = o(t)$, the space hierarchy theorem yields a language in $\mathrm{DSPACE}(n)$ outside $\mathrm{NTIME}(n)$, and its almost-everywhere form (Lemma 2.1) yields Theorem 1.

Sections 3 and 4 do not depend on Section 5, and they come first because they contain the new part. Section 2 collects preliminaries. Appendix A gives Kintali's construction explicitly and proves Proposition 3.3 and Lemma 4.5, Appendix B gives the counting algorithm as pseudocode, with the lifetime of every buffer, and Appendix C lists finite checks of the algebra.

1.3 Relation to prior work

Our counting algorithm is Kintali's [11], a simplification of the construction of OpenAI [13]. Kintali states his construction for 3-CNFs and for parameters that do not depend on N; letting his parameters grow with N gives time $2^{N/(\log N)^{1/4-o(1)}}$. Following his arguments, we prove the construction for constraints of any arity and for all parameters (Section 3 and Appendix A). The new part is its implementation in sublinear space (Section 4 and Appendix B), together with the choice of parameters, which also improves the running time to $2^{O(\kappa(N))}$, and the extension from clauses to constraints of logarithmic arity, which Section 5 needs. Apart from this, the paper uses only classical facts: the space hierarchy theorem, which we reprove in the form we need (Lemma 2.1), Hoeffding's inequality, the prime number theorem, and Chinese remaindering in logarithmic space.

Together with the sparsification lemma of Impagliazzo, Paturi and Zane [10], the paper therefore contains a self-contained refutation of the Exponential Time Hypothesis. For every $\varepsilon > 0$, that lemma writes a 3-CNF on n variables, in time $2^{\varepsilon n}\mathrm{poly}(n)$, as a disjunction of at most $2^{\varepsilon n}$ 3-CNFs in each of which every variable occurs $O_\varepsilon(1)$ times. Replacing every variable with d occurrences by d copies joined by a path of equality constraints turns each of them into a bounded-occurrence constraint system with $O_\varepsilon(n)$ variables, whose satisfiability Corollary 4.8 decides in space $O_\varepsilon(\kappa(n))$, and hence in time $2^{O_\varepsilon(\kappa(n))} = 2^{o(n)}$ (Section 2). Consequently, for every $\varepsilon > 0$ there is an algorithm that decides 3-SAT on n variables in time $2^{2\varepsilon n}$ for all large n.

All results are asymptotic, with very large constants. Section 7 discusses what remains to be checked, shows that the main theorem, like the separations of [9] and [14], does not relativize, and explains what does not follow.

2 Preliminaries

Machines. Machines are multitape Turing machines with a read-only input tape and K one-way infinite work tapes, and space counts work-tape cells. Every tape has a left endmarker in cell 0, which is never overwritten and from which no head moves left, and the input tape holds the input followed by a right endmarker, from which the input head does not move right. A configuration is *halting* if its state is accepting or rejecting; we assume that every other configuration has an applicable transition.

Nondeterminism. A nondeterministic machine has at most two applicable transitions in each configuration (every machine can be brought into this form with a constant-factor slowdown), and its *running time* $T(n)$ is the largest number of steps of a computation on an input of length n. If all its computations halt, it *decides* the set of inputs on which some computation accepts,

and then $T(n)$ is finite, by König's lemma. A *choice string* $w \in \{0,1\}^\ell$ determines a computation of length ℓ: at step i, if two transitions are applicable, bit w_i selects one, and otherwise w_i is ignored; a halted machine stays in its halting configuration. NTIME(T) is the class of languages decided by nondeterministic machines with running time at most T, and coNTIME(T) consists of their complements. Thus every computation, not only every accepting one, has length at most $T(n)$; for time-constructible T, such as $T(n) = n$, bounding only the accepting computations gives the same class.

Constraint systems. A *constraint system* F on N Boolean variables is a list of constraints e, each with a scope $I_e \subseteq [N]$ and a table $\gamma_e : \{0,1\}^{I_e} \to \{0,1\}$. An assignment $x \in \{0,1\}^N$ *satisfies* F if $\gamma_e(x|_{I_e}) = 1$ for every e, and

$$\#F = \sum_{x \in \{0,1\}^N} \prod_e \gamma_e(x|_{I_e})$$

is the number of satisfying assignments. The *arity* of F is the largest size of a scope, and F has *bounded occurrence* if every variable lies in at most three scopes. A 3-CNF in which every variable occurs in at most three clauses is a bounded-occurrence constraint system: each clause is a constraint whose scope is the set of its variables and whose table is its truth table, so that an empty clause has empty scope and table 0. A constraint system is *given in logarithmic space* by a deterministic transducer if the transducer, from an input of length polynomial in N, outputs N, the number of constraints, the scope of a given constraint and the value of its table at a given argument, in space $O(\log N)$, and the number of constraints is polynomial in N.

Lemma 2.1 (Space hierarchy, almost-everywhere form)**.** *Let $S(n) \geq \log_2 n$ be space-constructible. There is a language $L_S \in \mathrm{DSPACE}(S)$ such that every deterministic machine that decides L_S uses more than $c\,S(n)$ work cells on some input of length n, for some $c > 0$ and all large n. In particular, $L_S \notin \mathrm{DSPACE}(S')$ for every function S' with $\liminf_{n\to\infty} S'(n)/S(n) = 0$.*

This is the classical theorem of Stearns, Hartmanis and Lewis [17]; we include the proof because we use the almost-everywhere form.

Proof. Fix an encoding of deterministic machines by binary strings $\langle \mathcal{M} \rangle$ that lists the states, so that $\mathcal{M}$ has at most $|\langle \mathcal{M} \rangle|$ states. The machine $\mathcal{D}$ works as follows on an input w of length n. It rejects unless $w = \langle \mathcal{M} \rangle 10^j$ for a machine $\mathcal{M}$ and some $j \geq 0$. It marks $S(n)$ cells on each of its work tapes, which is possible in space $O(S(n))$ because S is space-constructible, rejects if $|\langle \mathcal{M} \rangle| > S(n)$, and otherwise copies $\langle \mathcal{M} \rangle$ to a work tape. Then it simulates $\mathcal{M}$ on w. Its own input head plays the role of the input head of $\mathcal{M}$; the current state of $\mathcal{M}$ is kept on a work tape; and the work tapes of $\mathcal{M}$, with a mark at the position of each head, are stored on tracks of one work tape, $c_{\mathcal{M}}$ cells of $\mathcal{D}$ for each cell of $\mathcal{M}$, where $c_{\mathcal{M}}$ depends only on the number of work tapes and the alphabet of $\mathcal{M}$. $\mathcal{D}$ rejects if the stored tapes would leave the $S(n)$ marked cells of that tape. A binary counter of $C_0 S(n)$ bits counts the simulated steps, and $\mathcal{D}$ rejects if it overflows. Here $C_0 = 3 + \lceil \log_2 |\Sigma| \rceil$, where Σ is the work alphabet of $\mathcal{D}$, which does not depend on $\mathcal{M}$. This is large enough: as long as the stored tapes fit, a configuration of $\mathcal{M}$ on w is determined by its state, the position of its input head, and the contents of the $S(n)$ marked cells that hold the stored tapes, head marks included; so, for $n > 4$, there are at most

$$|\langle \mathcal{M} \rangle|\,(n+2)\,|\Sigma|^{S(n)} \leq 2^{S(n)} \cdot 2^{S(n)+2} \cdot |\Sigma|^{S(n)} < 2^{C_0 S(n)}$$

such configurations, because $|\langle\mathcal{M}\rangle| \le S(n)$ and $\log_2 n \le S(n)$. If the simulated computation halts, $\mathcal{D}$ accepts if and only if $\mathcal{M}$ rejects. Altogether $\mathcal{D}$ uses space $O(S(n))$, and $L_S = L(\mathcal{D}) \in \mathrm{DSPACE}(S)$.

Let $\mathcal{M}$ decide L_S, and let n be so large that $n > \max(|\langle\mathcal{M}\rangle|, 4)$ and $S(n) \ge |\langle\mathcal{M}\rangle|$. Suppose that $\mathcal{M}$ uses at most $S(n)/c_{\mathcal{M}}$ work cells on every input of length n, and let $w = \langle\mathcal{M}\rangle 10^{n-|\langle\mathcal{M}\rangle|-1}$. On w the stored tapes fit, and the simulation ends before the counter overflows, because $\mathcal{M}$ halts, a halting deterministic computation never repeats a configuration, and fewer than $2^{C_0 S(n)}$ configurations have stored tapes that fit. Then $\mathcal{D}$ answers the opposite of $\mathcal{M}$ on w, which is impossible. Hence $\mathcal{M}$ uses more than $S(n)/c_{\mathcal{M}}$ work cells on some input of length n, for all such n. □

Composition and time. When a procedure that uses space s_1 reads its input through an oracle whose answers can be computed in space s_2, the two together run in space $O(s_1 + s_2)$, recomputing each answer when it is needed; we use this throughout without further mention. Such compositions can cost time, unavoidably in general [15], but we bound only space. A deterministic machine that halts and uses space $s \ge \log n$ never repeats a configuration, so it runs in time $2^{O(s)}$, and each of our space bounds S therefore comes with the time bound $2^{O(S)}$.

3 Kintali's construction

This section presents Kintali's construction [11] through the properties that the counting algorithm needs, and proves them in the generality we need: constraints of any arity, and parameters that grow with N. The explicit construction and the proof of Proposition 3.3 are in Appendix A.

The idea. Let Φ_e be the multilinear extension of the table of constraint e, so that $\#F = \sum_x \prod_e \Phi_e(x|_{I_e})$. The construction replaces every input x_v of every constraint e by a linear form $L_{e,v}(T; x)$ in the variables, whose coefficients are polynomials of degree less than R in a new variable T, with $L_{e,v}(0; x) = x_v$. Then $H(T) = \sum_x \prod_e \Phi_e((L_{e,v}(T; x))_v)$ is a polynomial with $H(0) = \#F$. The forms are chosen so that at each node $T = t \in [Q]$ the system decouples into r groups, up to integer linear measurements, each of which reads a single group (Section 3.2); for our parameters the values of all measurements take only $o(N/r)$ bits (Lemma 4.1). Fixing the values of the measurements turns the sum over x into a sum, over these values, of products of sums over single groups (Lemma 3.1). This computes the values $H(t)$, but H has degree less than $3NR$, far more than the number Q of nodes. What determines $H(0)$ is the Taylor expansion of H to order $J \approx 3NR/Q$ at every node, and these expansions have the same decoupled form, with an additional sum over sets of at most $J-1$ variables (Lemma 3.2). Section 3.3 records how the forms are obtained from two group schedules, which assign the variables and the constraints to groups at every node (Proposition 3.3). In Sections 3 and 4 the letter t always denotes a node and T the variable of the deformation; elsewhere they denote time bounds.

3.1 The counting polynomial

Let F be a bounded-occurrence constraint system on $[N]$ whose scopes are nonempty. Following [11, §2.1], add a dummy constraint, with scope $\{v\}$ and table constantly 1, for each variable v that lies in no scope. Let $\mathcal{F}$ be the set of constraints, and call a pair (e, v) with $v \in I_e$ an *incidence*; let M be the number of incidences. Each variable lies in between one and three scopes,

so $N \le M \le 3N$. For each constraint let

$$\Phi_e((X_v)_{v\in I_e}) = \sum_{a\in\{0,1\}^{I_e}} \gamma_e(a) \prod_{v\in I_e} \big(a_v X_v + (1-a_v)(1-X_v)\big)$$

be the multilinear extension of its table; it agrees with γ_e on Boolean arguments and has degree at most one in each variable. For a clause it is the polynomial used in [11]. Then

$$\#F = \sum_{x\in\{0,1\}^N} \prod_{e\in\mathcal{F}} \Phi_e((x_v)_{v\in I_e}). \tag{1}$$

3.2 Deformations and their Taylor data

Fix integers $r \ge 2$ and $Q, R, D \ge 1$. *Deformation data* consist of a form $L_{e,v}(T;X) = \sum_{i\in[N]} p_{e,v,i}(T) X_i$ for each incidence, with $p_{e,v,i} \in \mathbb{Q}[T]$, such that $L_{e,v}(0;X) = X_v$ and $\deg_T L_{e,v} \le R-1$, and, for each node $t \in [Q]$:

(D1) a partition $[N] = S_{t,1} \sqcup \cdots \sqcup S_{t,r}$ into groups with $|S_{t,g}| \le 2N/r$;

(D2) an assignment $\sigma_t : \mathcal{F} \to [r]$;

(D3) integer linear forms (*measurements*), each assigned to a group g and supported on $S_{t,g}$, each with coefficient ℓ_1-norm at most D;

(D4) for every incidence (e,v), a representation of $L_{e,v}(t;X)$ as a rational linear combination of the variables in $S_{t,\sigma_t(e)}$ and the measurements.

In words, at node t every deformed input $L_{e,v}(t;x)$ of constraint e is determined by the variables of its group $\sigma_t(e)$ and by the values of the measurements. Let

$$H(T) = \sum_{x\in\{0,1\}^N} \prod_{e} \Phi_e((L_{e,v}(T;x))_{v\in I_e}).$$

Then $H(0) = \#F$, and $\deg H \le M(R-1) < 3NR$ because each Φ_e has degree at most one in each of its $|I_e|$ inputs. Put $q = \lfloor 3NR/Q \rfloor$ and $J = q+1$, so that $QJ > \deg H$.

Lemma 3.1 (Grouped sums [11, eq. (2.4)])**.** *Fix $t \in [Q]$, $E \subseteq [N]$ and polynomials $b_{e,v}(z)$ not depending on x. For a group g let C_g be the integer matrix of the measurements assigned to group g. For $y = (y_1, \dots, y_r)$, where y_g is a vector with one entry in $\{-D, \dots, D\}$ for each measurement of group g, let $\lambda_{e,v}(x_g; y)$ be the representation (D4) of $L_{e,v}(t;x)$ with the measurement values replaced by y, where $g = \sigma_t(e)$. Then, as polynomials in z,*

$$\sum_{\substack{x\in\{0,1\}^N \\ x_i=1\,(i\in E)}} \prod_{e} \Phi_e((L_{e,v}(t;x) + b_{e,v}(z))_v) = \sum_{y} \prod_{g=1}^{r} \sum_{\substack{x_g\in\{0,1\}^{S_{t,g}} \\ C_g x_g = y_g,\ x_i=1\,(i\in E\cap S_{t,g})}} A_g(x_g; y, z),$$

where $A_g(x_g; y, z) = \prod_{e:\sigma_t(e)=g} \Phi_e((\lambda_{e,v}(x_g;y) + b_{e,v}(z))_v)$.

Proof. Every $x \in \{0,1\}^N$ determines the tuple y of its measurement values, $y_g = C_g x_g$ with $x_g = x|_{S_{t,g}}$, and these values lie in $\{-D, \dots, D\}$ by (D3). Group the sum on the left by y. For an x with measurement values y, (D4) gives $L_{e,v}(t;x) = \lambda_{e,v}(x_g;y)$ with $g = \sigma_t(e)$, so the product over e equals $\prod_g A_g(x_g; y, z)$; and the conditions $C_g x_g = y_g$ and $x_i = 1$ for $i \in E$ are conditions on the groups separately. So, for fixed y, the sum over x is the product over g of the sums over x_g. □

Kintali states this identity modulo z^J; the same argument proves it in $\mathbb{Q}[z]$, which is the form we use.

The second identity writes the Taylor expansion of H at a node to order J in the decoupled form of Lemma 3.1: each term fixes a small set E of variables to 1 and adds to every form a correction $b^U_{e,v}(z)$ that depends on a subset $U \subseteq E$. The terms are indexed by *marked supports*, vectors $\eta \in \{0,+,-\}^N$. Write $E(\eta) = \{i : \eta_i \neq 0\}$, $U(\eta) = \{i : \eta_i = +\}$, $|\eta| = |E(\eta)|$ and $\eta^- = E(\eta) \setminus U(\eta)$; marked supports correspond to pairs $U \subseteq E$.

Lemma 3.2 (Low-order Taylor data [11, §2.3, eq. (2.9)])**.** *For $t \in [Q]$ and $U \subseteq [N]$ let $b^U_{e,v}(z) = L_{e,v}(t+z; \mathbf{1}_U) - L_{e,v}(t; \mathbf{1}_U)$, and define the polynomial*

$$Y_t(z) = \sum_{|\eta| \le q} (-1)^{|\eta^-|} \sum_{\substack{x \in \{0,1\}^N \\ x_i = 1 \ (i \in E(\eta))}} \prod_e \Phi_e\big((L_{e,v}(t;x) + b^{U(\eta)}_{e,v}(z))_v\big). \tag{2}$$

Then $Y_t(z) \equiv H(t+z) \pmod{z^J}$, and $\deg_z Y_t \le M(R-1) < 3NR$.

Proof. The congruence is [11, eq. (2.9)], written with marked supports; we give the short argument, since we use the untruncated polynomial, which Section 4.2 evaluates at points of $\mathbb{F}_p^\times$, while Kintali truncates it modulo z^J. Fix $x \in \{0,1\}^N$ and let $X = (X_1, \dots, X_N)$ be indeterminates. Each correction $L_{e,v}(t+z;X) - L_{e,v}(t;X)$ is linear in X with coefficients divisible by z. Hence in

$$F_x(X,z) = \prod_e \Phi_e\big((L_{e,v}(t;x) + L_{e,v}(t+z;X) - L_{e,v}(t;X))_v\big)$$

every monomial of degree d in X carries the factor z^d, and $H(t+z) = \sum_x F_x(x,z)$. For $E \subseteq [N]$ put $\Delta_E F_x = \sum_{U \subseteq E} (-1)^{|E|-|U|} F_x(\mathbf{1}_U, z)$. A monomial in X contributes to $\Delta_E F_x$ only if its set of variables is exactly E, so $z^{|E|}$ divides $\Delta_E F_x$; and Möbius inversion on the Boolean lattice gives $F_x(x,z) = \sum_{E \subseteq \{i : x_i = 1\}} \Delta_E F_x$. Modulo z^J only the sets with $|E| \le q$ remain. Since $F_x(\mathbf{1}_U, z) = \prod_e \Phi_e\big((L_{e,v}(t;x) + b^U_{e,v}(z))_v\big)$, summing over x and writing the pairs $U \subseteq E$ as marked supports gives $H(t+z) \equiv Y_t(z) \pmod{z^J}$. For the degree bound, each $b^U_{e,v}$ has degree at most $R-1$ in z and each Φ_e is multilinear in its $|I_e|$ inputs, so $\deg_z Y_t \le \sum_e |I_e|(R-1) = M(R-1)$. □

3.3 The construction

Group schedules are maps $a : [N] \to [r]^Q$, for the variables, and $d : \mathcal{F} \to [r]^Q$, for the constraints. Incidence (e,v) gets the *label* $u(e,v) = ((a_v(t), d_e(t)))_{t \in [Q]} \in ([r] \times [r])^Q$. Let c_u be the number of incidences with label u, and $n_{t,g} = |\{v : a_v(t) = g\}|$. Let $D = r^{2Q}$ be the number of labels. The construction stores each incidence at the coordinate given by its label, in one of $\max_u c_u$ copies of the space of labels, and each measurement reads one copy; so the coefficient ℓ_1-norm of a measurement is at most D, as in (D3), and the first condition in (3) below keeps the number of copies, and hence of measurements, small (Appendix A).

Proposition 3.3 (Kintali's construction [11, Proposition 3.1])**.** *Let $r \ge 2$, $Q \ge 1$ and $R \ge 2$ be integers. With every pair of group schedules, Appendix A associates forms $L_{e,v}$ and, at each node t, the groups $S_{t,g} = \{v : a_v(t) = g\}$, the assignment $\sigma_t(e) = d_e(t)$ and measurements. If the group schedules satisfy*

$$c_u \le 2M/D \ \text{ for every label } u \qquad \text{and} \qquad n_{t,g} \le 2N/r \ \ (t \in [Q],\, g \in [r]), \tag{3}$$

then these are deformation data, with at most $4Mr\, e^{-R^2/(8Q)}$ measurements at each node.

symbol	meaning	size
$r = s = 2^k$	number of groups	$\asymp \sqrt{\log N}/\log\log N$
$Q = s^2 k$	number of nodes	$\asymp \log N/\log\log N$
$R = 8sk$	the forms have degree less than R in T	$\asymp \sqrt{\log N}$
$D = r^{2Q}$	number of labels; bound on the measurement values	$\leq N^{1/8}$
$q = \lfloor 3NR/Q \rfloor$	largest size $\lvert\eta\rvert$ of a marked support	$\leq 24N/s$
$J = q + 1$	order of the Taylor data	
p	prime modulus	$4N^2 < p \leq N^{c_0}$

Table 1: The parameters of the counting algorithm; k is the largest integer with $16 \cdot 4^k k^2 \leq \lfloor \log_2 N \rfloor$.

Kintali states this as [11, Proposition 3.1] for 3-CNFs and for parameters that do not depend on N, with additional hypotheses that are used only to construct group schedules satisfying (3), which Lemma 4.4 below does in our setting. Appendix A proves the proposition as stated, following his argument: clauses enter it only through their incidences, and $M \leq 3N$ follows from bounded occurrence alone.

4 Counting in sublinear space

Kintali's algorithm evaluates the decoupled sums of Lemmas 3.1 and 3.2 at every node, stores the resulting Taylor coefficients, and recovers $H(0)$ by Hermite interpolation. This section turns it into an algorithm that runs in space $O(\kappa(N))$. We fix the parameters first (Section 4.1 and Table 1). Section 4.2 replaces Hermite interpolation by an identity that recovers $H(0)$ from the values, at the points of $\mathbb{F}_p^\times$, of the untruncated polynomials Y_t of Lemma 3.2, which Lemma 3.1 evaluates without computing their coefficients, and Section 4.3 combines it with Lemmas 3.1 and 3.2 into a single formula for $\#F \bmod p$. The rest of the section shows that this formula can be evaluated by nested loops in space $O(\kappa(N))$: Section 4.4 generates the group schedules from a short seed, Section 4.5 computes the coefficients of the construction on demand and writes the supports compactly, and Section 4.6 puts the pieces together.

Throughout this section, c_0 and c_1 are constants, F is a bounded-occurrence constraint system on N variables with arity $\Delta \leq c_1 \log_2 N$, and p is a prime with $4N^2 < p \leq N^{c_0}$. The system is given in logarithmic space by a fixed transducer (Section 2); its input has length polynomial in N, so a pointer into it takes $O(\log N)$ bits. We may assume that all scopes are nonempty: a constraint with empty scope is a constant factor of $\#F$. For the parameters below $p > \max(Q, R)$, and every rational number in the construction has a denominator that is a product of powers of 2 and of nodes $t \leq Q$ (Appendix A). Everything can therefore be reduced modulo p, and Lemmas 3.1 and 3.2 remain true after reduction.

4.1 Parameters

Let k be the largest integer with $16 \cdot 4^k k^2 \leq \lfloor \log_2 N \rfloor$. For N so large that $k \geq 6$, set

$$s = 2^k, \qquad r = s, \qquad Q = s^2 k, \qquad R = 8sk, \qquad D = r^{2Q} = 2^{2s^2 k^2}. \tag{4}$$

In terms of N, there are $r \asymp \sqrt{\log N}/\log\log N$ groups and $Q \asymp \log N/\log\log N$ nodes, the forms have degree less than $R \asymp \sqrt{\log N}$ in T, and $q \leq 24N/s$. For smaller N, brute force suffices.

Lemma 4.1. *For N above a threshold that depends only on c_1, the parameters (4) satisfy $4D^2 \le M$, $(\Delta+1)r^{-Q} \le 1$ and $2Qr^2 \le N$. Moreover $D \le N^{1/8}$, $q \le 24N/s$, there are at most $12N/s^7$ measurements at each node, their values can be written with at most $30Nk^2/s^5$ bits, and $Nk/s = O(\kappa(N))$.*

Proof. From $16s^2k^2 \le \log_2 N$ we get $D = 2^{2s^2k^2} \le N^{1/8}$, so $4D^2 \le 4N^{1/4} \le N \le M$, and $2Qr^2 = 2s^4k \le N$. The maximality of k gives $\log_2 N < 16\cdot 4^{k+1}(k+1)^2 = 64s^2(k+1)^2 \le 100s^2k^2$, as $k \ge 6$; so $r^Q = 2^{s^2k^2} \ge N^{1/100}$ and $(\Delta+1)r^{-Q} \le (c_1\log_2 N + 1)N^{-1/100} \le 1$ once N is large enough in terms of c_1. By definition $q \le 3NR/Q = 24N/s$. Since $R^2/(8Q) = 8k$, Proposition 3.3 gives at most $4Mre^{-8k} \le 12Ns\cdot 2^{-8k} = 12N/s^7$ measurements per node, each with a value in $\{-D,\dots,D\}$, written with $\lceil \log_2(2D+1)\rceil \le 2s^2k^2+2$ bits, and $12N(2s^2k^2+2)/s^7 \le 30Nk^2/s^5$. Finally, $s > \sqrt{\log_2 N}/(8(k+1))$ by the maximality of k, and $k \le \log_2\log_2 N$; hence $k/s = O((\log\log N)^2/\sqrt{\log N})$. □

Kintali states his result for fixed parameters; letting s grow with N, his choice $Q = s^4$, $R = s^3$ [11, §4] gives the running time $2^{N/(\log N)^{1/4-o(1)}}$. With (4), $R/\sqrt{Q} = 8\sqrt{k}$ is just large enough for the number of measurements in Proposition 3.3, which carries the factor $e^{-R^2/(8Q)} = e^{-8k}$, to be small (see Section 7).

4.2 Recovering $H(0)$ without interpolation

Kintali recovers $H(0)$ from the Taylor coefficients $[z^i]Y_t$, $i < J$, by Hermite interpolation. There are $QJ > 3NR$ of them, too many to store. The following identity obtains $H(0)$ from values of the polynomials Y_t instead. For $t \in [Q]$ and $p > Q$ let

$$G_t(z) = \frac{1}{t+z}\prod_{t'\in[Q]\setminus\{t\}} \frac{1}{(t-t'+z)^J} \in \mathbb{F}_p[[z]],$$

a power series because t and $t-t'$ are nonzero modulo p, and let $G_t^{<J}$ be the polynomial formed by its terms of degree less than J.

Lemma 4.2. *Let p be a prime and $Q, J \ge 1$ with $Q < p$ and $J < p$. Let $H \in \mathbb{F}_p[X]$ with $\deg H < QJ$, and let $Y_1,\dots,Y_Q \in \mathbb{F}_p[z]$ satisfy $Y_t(z) \equiv H(t+z) \pmod{z^J}$ and $\deg Y_t \le p-2$. With $\Omega(X) = \prod_{t'=1}^{Q}(X-t')^J$,*

$$H(0) = \Omega(0)\sum_{t=1}^{Q}\sum_{z\in\mathbb{F}_p^\times} z^{p-J}\, G_t^{<J}(z)\, Y_t(z). \tag{5}$$

The coefficients of $G_t^{<J}$ can each be computed in space $O((1+\log Q)\log(pJ))$.

Conceptually, (5) is the residue theorem for the rational function $H(X)/(X\Omega(X))$: its residue at 0 is $H(0)/\Omega(0)$, its residues at the nodes are determined by the Taylor data, and the sum over $\mathbb{F}_p^\times$ extracts the coefficient that is needed. We give a direct proof.

Proof. Partial fractions. Since $p > Q$, the factors $(X-t')^J$ are pairwise coprime, and $\deg H < \deg\Omega$. Hence $H/\Omega = \sum_{t=1}^{Q}\sum_{j=1}^{J} c_{t,j}(X-t)^{-j}$ for some $c_{t,j} \in \mathbb{F}_p$, and evaluating at $X = 0$ gives $H(0) = \Omega(0)\sum_{t,j} c_{t,j}(-t)^{-j}$.

Taylor data at t. Fix t and put $X = t+z$. In the ring $\mathbb{F}_p((z))$ of Laurent series, $H(t+z)/\Omega(t+z) = \sum_j c_{t,j}z^{-j} + B_t(z)$ with $B_t \in \mathbb{F}_p[[z]]$, because the terms for $t' \ne t$ have no pole at $z = 0$.

Multiply by $1/(t+z) = \sum_{i\geq 0}(-1)^i t^{-i-1} z^i$ and take the coefficient of z^{-1}. On the right this gives $\sum_j c_{t,j}(-1)^{j-1}t^{-j} = -\sum_j c_{t,j}(-t)^{-j}$. On the left, since $\Omega(t+z) = z^J \prod_{t'\neq t}(t-t'+z)^J$, we have $H(t+z)/((t+z)\Omega(t+z)) = z^{-J}H(t+z)G_t(z)$, whose coefficient of z^{-1} is $[z^{J-1}](H(t+z)G_t(z))$. This coefficient depends only on the terms of degree less than J of the two factors, so it equals $[z^{J-1}](Y_t G_t^{<J})$. Hence $\sum_j c_{t,j}(-t)^{-j} = -[z^{J-1}](Y_t G_t^{<J})$.

Projector. For $P \in \mathbb{F}_p[z]$ and $0 \leq i < p-1$ we have $\sum_{z\in\mathbb{F}_p^\times} z^{p-1-i}P(z) = -\sum_\ell [z^\ell]P$, the sum over all $\ell \equiv i \pmod{p-1}$, because $\sum_{z\in\mathbb{F}_p^\times} z^{\ell'}$ equals -1 if $(p-1) \mid \ell'$ and 0 otherwise. Take $P = Y_t G_t^{<J}$ and $i = J-1 < p-1$. Since $\deg P \leq \deg Y_t + J - 1 < (J-1)+(p-1)$, the only index in this sum that can contribute is $\ell = J-1$, so $[z^{J-1}](Y_t G_t^{<J}) = -\sum_{z\in\mathbb{F}_p^\times} z^{p-J} G_t^{<J}(z) Y_t(z)$. Combining the three steps proves (5).

Computation. G_t is a product of Q series of the form $(c+z)^{-m}$ with $m \in \{1, J\}$, and $[z^i](c+z)^{-m} = (-1)^i \binom{m+i-1}{i} c^{-m-i}$, where the binomial coefficient is computed modulo p from the multiplicative formula (for $i < J < p$, $i!$ is invertible modulo p). A coefficient of a product of series is computed by divide and conquer: split the factors into two halves and sum, over the ways of splitting the degree, the products of the corresponding coefficients of the two halves. The recursion has depth $\lceil \log_2 Q \rceil$, and each level stores a degree, the current split of it, the value of the left half and an accumulator, in $O(\log(pJ))$ bits. □

4.3 The master formula

We now combine Lemmas 3.1 and 3.2 with Lemma 4.2. Recall the notation of Lemma 3.1: at node t, C_g is the matrix of the measurements of group g, $y = (y_1, \ldots, y_r)$ is a tuple of values of all measurements, and $\lambda_{e,v}(x_g; y)$ is the deformed input $L_{e,v}(t; x)$ computed from its representation (D4), with the measurement values replaced by y.

Proposition 4.3 (Master formula)**.** *Let the parameters be as in (4), let the group schedules satisfy (3) (such group schedules exist by Lemma 4.4), and take the deformation data of Proposition 3.3. Then, modulo p,*

$$\#F \equiv \Omega(0) \sum_{t=1}^{Q} \sum_{z\in\mathbb{F}_p^\times} z^{p-J} G_t^{<J}(z) \sum_{|\eta|\leq q} (-1)^{|\eta^-|} \sum_y \prod_{g=1}^{r} \sum_{x_g} A_g^\eta(x_g; y, z),$$

where $y = (y_1, \ldots, y_r)$ ranges over the tuples of Lemma 3.1, x_g over the assignments to $S_{t,g}$ with $C_g x_g = y_g$ and $x_i = 1$ for $i \in E(\eta) \cap S_{t,g}$, and $A_g^\eta(x_g; y, z) = \prod_{e:\sigma_t(e)=g} \Phi_e((\lambda_{e,v}(x_g; y) + b_{e,v}^{U(\eta)}(z))_v)$.

Proof. By Lemma 4.1 and Proposition 3.3 we have deformation data, and Lemmas 3.1 and 3.2 hold modulo p. By Lemma 3.1, with $E = E(\eta)$ and $b = b^{U(\eta)}$, the sum over y is the inner sum over x in (2); so the sum over η is $Y_t(z)$ mod p. By Lemma 3.2, $Y_t(z) \equiv H(t+z) \pmod{z^J}$ and $\deg Y_t < 3NR$; also $\deg H < 3NR < QJ$, and $3NR + J < 4N^2 < p$, so $J < p$ and $\deg Y_t \leq p-2$. Lemma 4.2 and $H(0) = \#F$ give the claim. □

4.4 Group schedules from a short seed

Since $r = 2^k$, an element of $[r]^Q$ is a string of Qk bits, k bits per node. Let $\ell = \lceil \log_2(N + |\mathcal{F}| + 1) \rceil$, and identify the variables and the constraints with distinct nonzero vectors in $\mathrm{GF}(2)^\ell$ by

binary representation. A *seed* is a tuple $(\Gamma, \theta, \Gamma', \theta')$ of matrices $\Gamma, \Gamma' \in \mathrm{GF}(2)^{Qk \times \ell}$ and vectors $\theta, \theta' \in \mathrm{GF}(2)^{Qk}$. It defines the group schedules

$$a_v = \Gamma v + \theta, \qquad d_e = \Gamma' e + \theta'.$$

A seed has $2Qk(\ell+1) = O(\log^2 N)$ bits, because $Qk = s^2k^2 \le \log_2 N$.

Lemma 4.4. *(i) Some seed yields group schedules satisfying (3).*

(ii) The first seed in lexicographic order whose group schedules satisfy (3) can be found in space $O(\log^2 N)$.

(iii) Given a seed, the number $|\mathcal{F}|$ of constraints, including the dummy constraints of Section 3, the values $a_v(t)$, $d_e(t)$ and $u(e,v)$, the counts c_u and $n_{t,g}$, whether $v \in S_{t,g}$, and the position of v in $S_{t,g}$ can be computed in space $O(\log^2 N)$.

Proof. (i) Choose the seed uniformly at random. For distinct variables $v \neq v'$ the vector $v + v'$ is nonzero, so $\Gamma(v+v')$ is uniform on $\mathrm{GF}(2)^{Qk}$; since θ is uniform and independent of Γ, the pair $(\Gamma v + \theta, \Gamma(v+v'))$ is uniform, and hence so is $(a_v, a_{v'}) = (\Gamma v + \theta,\ \Gamma v + \theta + \Gamma(v+v'))$. Thus the variable schedule is pairwise independent with uniform values. The same holds for the constraint schedule, and the two group schedules are independent.

Let $\Theta = \sum_u (c_u - M/D)^2$ and $\Theta' = \sum_{t,g} (n_{t,g} - N/r)^2$. Expanding as in [11, Lemma 3.4], $\Theta = \sum \mathbf{1}[u(e,v) = u(e',v')] - M^2/D$, where the sum is over ordered pairs of incidences, equal pairs included. Equal labels have probability 1 for the M equal pairs; probability r^{-Q} for the n_1 pairs of distinct incidences that share their variable or their constraint (they cannot share both), since then the labels agree if and only if the two constraints, or the two variables, get the same group schedule; and probability $\Pr[a_v = a_{v'}] \Pr[d_e = d_{e'}] = 1/D$ for the remaining $n_0 \le M^2$ pairs. Each incidence shares its variable with at most two other incidences and its constraint with at most $\Delta - 1$, so $n_1 \le (\Delta+1)M$, and

$$\mathbb{E}\Theta = M + n_1 r^{-Q} + \frac{n_0 - M^2}{D} \le M\bigl(1 + (\Delta+1)r^{-Q}\bigr) \le 2M$$

by Lemma 4.1. Similarly $\mathbb{E}\Theta' = QN(1 - 1/r) \le QN$, using only pairwise independence. Hence $\mathbb{E}[\Theta/(2M) + \Theta'/(QN)] \le 2$, and some seed has $\Theta \le 4M$ and $\Theta' \le 2QN$. For this seed $c_u \le M/D + 2\sqrt{M} \le 2M/D$, because $4D^2 \le M$, and $n_{t,g} \le N/r + \sqrt{2QN} \le 2N/r$, because $2Qr^2 \le N$.

(ii) Run through the seeds in lexicographic order. For each seed, compute the counts c_u one label at a time (there are $D \le N^{1/8}$ labels) and the counts $n_{t,g}$ one pair (t,g) at a time, each by streaming over the incidences or the variables with a counter of $O(\log N)$ bits, and stop at the first seed that satisfies (3). Part (iii) is immediate from the definitions. □

4.5 Computing the terms and writing the supports

Lemma 4.5. *Fix a seed satisfying (3). Each of the following can be computed in space $O(\log^2 N)$, with no stored tables:*

(a) modulo p, the coefficients of the representation (D4) of $L_{e,v}(t; X)$, for an incidence (e,v) and a node t;

(b) the value of a measurement at x, an integer in $\{-D, \dots, D\}$ computed exactly, where x is given by an oracle;

(c) modulo p, the correction $b^U_{e,v}(z)$ for $z \in \mathbb{F}_p$, where U is given by a membership oracle.

The proof, in Appendix A, uses an explicit formula for the entries of the matrix A of Appendix A, so that no matrix arithmetic and no preprocessing is needed.

Writing the supports. Divide $[N]$ into $\lceil N/s \rceil$ consecutive segments of at most $s = 2^k$ positions. The *code* of a marked support η treats the segments in order: for each segment it writes, for every i in the segment with $\eta_i \neq 0$, in increasing order, a bit 1, the offset of i in its segment in k bits, and the bit $\mathbf{1}[\eta_i = +]$; and then it writes a bit 0. Thus $|\mathrm{code}(\eta)| = \lceil N/s \rceil + (k+2)|\eta|$. Reading a code from the left is deterministic (a bit 0 ends a segment, and a bit 1 is followed by the $k+1$ bits of an entry), so the marked supports with $|\eta| \leq q$ are enumerated, each exactly once, by running through all binary strings of length at most $\Lambda = \lceil N/s \rceil + (k+2)q$ and keeping those that consist of exactly $\lceil N/s \rceil$ segments, with offsets strictly increasing within each segment and lying inside it, and with at most q entries. Given $\mathrm{code}(\eta)$ and $i \in [N]$, the value η_i is found by reading the code, in space $O(\log N)$.

4.6 The algorithm

Theorem 4.6. *There is a deterministic algorithm that, given p and the input of the transducer that gives F, outputs $\#F \bmod p$ in space $O(\kappa(N))$, with the constant depending on c_0, c_1 and the transducer.*

Proof. If N is below the threshold of Lemma 4.1, use brute force. Otherwise compute the parameters, find the seed of Lemma 4.4, and evaluate the right side of Proposition 4.3 by nested loops over t, z, η, y, g and x_g, accumulating sums and products modulo p, with η enumerated through its code. The conditions on x_g are checked with Lemma 4.5(b) and the code of η. Each factor of A_g^η is a value $\Phi_e(\xi)$ at the point with coordinates $\xi_v = \lambda_{e,v}(x_g; y) + b_{e,v}^{U(\eta)}(z)$, which are computed from x_g, y and Lemma 4.5(a),(c), reading $U(\eta)$ from the code; and $\Phi_e(\xi)$ is the sum, over the $2^{|I_e|} \leq N^{c_1}$ rows a of the table, of $\gamma_e(a) \prod_{v \in I_e} (a_v \xi_v + (1 - a_v)(1 - \xi_v))$, computed in space $O(\log^2 N)$. The coefficients of $G_t^{<J}$ come from Lemma 4.2.

Besides $O(\log^2 N)$ bits for the seed, for a coefficient of $G_t^{<J}$ and for evaluating the terms, the algorithm stores only the current values of the loop variables; the product over g is formed one group at a time, so only one assignment x_g is stored:

loop variable	range	bits
t, z, g	$[Q]$, $\mathbb{F}_p^\times$, $[r]$	$O(\log N)$
$\mathrm{code}(\eta)$	marked supports with $\lvert\eta\rvert \leq q$	$\leq \Lambda \leq (24k + 49)N/s + 1$
y	measurement values at node t	$\leq 30Nk^2/s^5$ (Lemma 4.1)
x_g	assignments to the group $S_{t,g}$	$\leq 2N/s$

By Lemma 4.1 the total is $O(Nk/s + \log^2 N) = O(\kappa(N))$. All subroutines read the loop variables $\mathrm{code}(\eta)$, y and x_g in place on the work tape, through pointers of $O(\log N)$ bits, and never copy them. Appendix B gives the algorithm as pseudocode, with the writer and the lifetime of each of these three buffers. □

Remark 4.7 (Integer tables)**.** The algorithm of Theorem 4.6 uses the tables only through their residues modulo p, and neither its proof nor the results it relies on use that the tables take values in $\{0, 1\}$. Hence, for integer-valued tables $\gamma_e : \{0,1\}^{I_e} \to \mathbb{Z}$, given modulo p, the same algorithm computes $\sum_x \prod_e \gamma_e(x|_{I_e}) \bmod p$ in the same space.

Corollary 4.8. *Whether a bounded-occurrence constraint system on N variables, given as above, is satisfiable can be decided in space $O(\kappa(N))$.*

Proof. For N below a constant threshold use brute force. Otherwise $0 \leq \#F \leq 2^N$, and the interval $(4N^2, 8N^2]$ contains at least N primes, by the prime number theorem, whose product

exceeds 2^N. Hence $\#F > 0$ if and only if $\#F \not\equiv 0$ modulo one of them. Find these primes by trial division and apply Theorem 4.6 to each, with $c_0 = 3$. □

Proof of Theorem 3. The counting statement is Theorem 4.6. Consider a 3-CNF with N variables and $O(N)$ clauses, written out as a list of clauses, in which every variable occurs in at most three clauses. Every nonempty clause contains a variable, so there are at most $3N$ nonempty clauses, and the hypothesis on the number of clauses only bounds the number of empty clauses. If there is an empty clause, the formula is unsatisfiable, which a scan of the input detects. Otherwise the formula, with each clause as a constraint whose scope is the set of its variables and whose table is its truth table, is a bounded-occurrence constraint system of arity at most 3, given in logarithmic space by the transducer that reads the list of clauses (Section 2), and Corollary 4.8 applies. □

5 From computations to constraint systems

We now represent the computations of a nondeterministic machine by bounded-occurrence constraint systems, using the block decomposition of Hopcroft, Paul and Valiant [9]. Each interval of the computation is a single constraint with its own choice bits, and only the work heads are scheduled.

Intervals, blocks and head schedules. Fix a nondeterministic machine $\mathcal{M}$ with K work tapes, an input x and an integer $t \geq \max(|x|, 4)$, and put $b = \lceil \log_2 t \rceil$, $m = \lceil t/b \rceil$ and $\bar{t} = mb$, so that $t \leq \bar{t} < 2t$. We divide the first $\bar{t}$ steps of a computation into m *intervals* of b steps, and every work tape into *blocks* of b consecutive cells, block 0 starting at cell 0. During an interval a work head moves at most b cells, so if it starts the interval in block B, it stays within blocks $B-1$, B and $B+1$ and ends the interval in one of them. A *head schedule* is an array $\pi \in \{-1, 0, 1\}^{K \times m}$ in which, for every work tape j, all partial sums $\pi_{j,1} + \cdots + \pi_{j,i}$ are nonnegative. The head schedule of a computation records, for every interval i and every work tape j, the change $\pi_{j,i}$ of the block of head j during interval i; it is a head schedule, since no head visits a negative cell. Each choice string determines a computation, and hence exactly one head schedule. Given π, the block of each work head at the start of each interval is a partial sum, computable in space $O(\log t)$. The *window* of a head in an interval consists of the blocks $B-1$, B and $B+1$ that have nonnegative index, where B is the block of the head at the start of the interval.

Lemma 5.1. *There is a constant C, depending only on $\mathcal{M}$, such that for every $\chi \in \{\mathrm{acc}, \mathrm{rej}, \mathrm{run}\}$ and every head schedule π there is a constraint system $F^{\chi}_{\pi} = F^{\chi}_{x,t,\pi}$ on between t and Ct variables, in which every variable lies in at most two scopes and every scope has at most $C \log_2 t$ variables, with the following properties.*

(a) A deterministic transducer that depends only on $\mathcal{M}$, given x, t, χ and π, outputs the number of variables of F^{χ}_{π} and its number m of constraints; given also the index of a constraint, it outputs its scope, and, given also an argument, the value of its table there; all in space $O(\log t)$. Thus F^{χ}_{π} is given in logarithmic space, from an input of length $O(t)$.

(b) $\sum_{\pi} \#F^{\chi}_{\pi}$ is the number of choice strings $w \in \{0,1\}^{\bar{t}}$ such that, after $\bar{t}$ steps, the computation of $\mathcal{M}$ on x determined by w is in an accepting configuration (if $\chi = \mathrm{acc}$), in a rejecting configuration (if $\chi = \mathrm{rej}$), or not in a halting configuration (if $\chi = \mathrm{run}$).

If the constraints also require every choice bit to be 0 at a step with a single applicable transition or after halting, then (b) holds with computations of length $\bar{t}$ in place of choice strings.

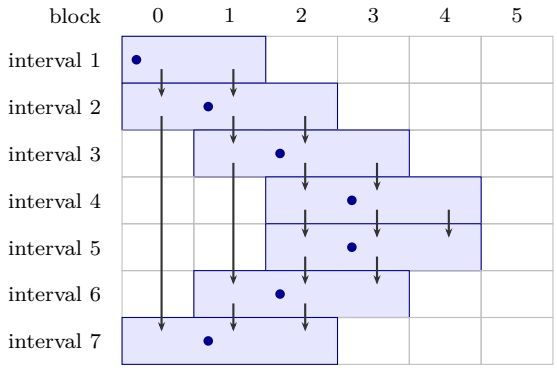


Figure 1: The windows of one work head during seven intervals, for a head schedule whose entries for this tape begin with $(+1, +1, +1, 0, -1, -1)$. Row i shades the window of interval i, and the dot marks the block of the head at the start of the interval. The contents of the blocks are variables of the constraint system: interval i reads the latest version of each block in its window and writes a new version. An arrow joins the interval that writes a version to the only interval that reads it, the next interval whose window contains the block; in between, the block keeps its contents. The first interval whose window contains a block reads its initial contents, which are constants. Hence every variable lies in at most two scopes.

Proof. Interval constraints. The *entering data* of an interval are the state, the position of the input head, the offsets of the work heads within their blocks, and the contents of the blocks in the windows of the work heads at the start of the interval; the *leaving data* are the state, the input-head position, the offsets, and the contents of the same blocks at its end. They are encoded in binary with fixed lengths, so they have $O(b)$ bits each, as the input head stays within the first $\bar{t} + 1 < 2t + 1$ cells. The constraint of interval i has as arguments the entering data, b choice bits u_i, and the leaving data. It is satisfied if the b steps of $\mathcal{M}$ that start from the entering data, read the input x and use the choice bits u_i produce the leaving data and change the blocks of the work heads as prescribed by π; for the last interval we also require that the final state be accepting ($\chi = \text{acc}$), rejecting ($\chi = \text{rej}$), or neither ($\chi = \text{run}$). Invalid encodings do not satisfy it. It is evaluated by simulating b steps on the $O(b)$ work cells concerned, reading input symbols from x, in space $O(\log t)$.

The constraint system. The variables are the choice bits and the bits of the leaving data of all intervals; there are at least $mb \geq t$ and at most Ct of them, for a suitable C. The entering state, input-head position and offsets of interval i are the leaving ones of interval $i - 1$, or the initial ones if $i = 1$. The entering contents of a block are the leaving contents of the last earlier interval in whose windows the block lies; if there is no such interval, they are its initial contents (the endmarker and blanks), which are substituted into the table as constants. Thus every choice bit lies in one scope, and every other variable lies in the scope of the constraint that produces it and of at most one constraint that reads it: for block contents, the next interval whose windows contain the block (Figure 1); for the state, the input-head position and the offsets, the next interval.

Counting. Let x' be a satisfying assignment, and let $w = u_1 \cdots u_m$ be its choice bits. By induction over i, the entering data of interval i are the true data of the computation determined by w after $(i - 1)b$ steps, and this computation has followed π so far. For $i = 1$ the entering data are the initial ones. If the claim holds for i, the constraint of interval i forces its leaving data

to be the true data after ib steps and the change of blocks to agree with π; and the entering data of interval $i+1$ are true because a block keeps its contents between two visits. Hence x' is determined by w, and the computation determined by w has head schedule π and meets the final requirement. Conversely, for every such w the true data give a satisfying assignment. Therefore $\#F_\pi^\chi$ is the number of these choice strings that have head schedule π, and summing over π proves (b). In particular, F_π^χ has no satisfying assignment if no computation has head schedule π. With the additional requirement on the choice bits, choice strings correspond bijectively to computations, which proves the last statement.

Locality. The scope of a constraint and the source of each of its arguments are determined by π and by the last earlier visit to each block, and the index of a variable and the number of variables are sums of lengths of data, all computable in space $O(\log t)$; the tables are evaluated as above. This proves (a), with C depending only on $\mathcal{M}$. □

6 Proofs of the main results

We now combine the counting algorithm, through Corollary 4.8, with Lemma 5.1.

Theorem 6.1. *Let $\mathcal{M}$ be a nondeterministic machine that decides a language L, with running time $T(n)$. A deterministic machine decides L, and its complement, in space $O(\kappa(T^*(n)))$, where $T^*(n) = \max(T(n), n)$. In particular* $\mathrm{NTIME}(T) \cup \mathrm{coNTIME}(T) \subseteq \mathrm{DSPACE}(\kappa(T))$ *for every $T(n) \geq n$.*

Proof. Let x be an input of length n. For an integer $t \geq \max(n, 4)$ and $\chi \in \{\mathrm{acc}, \mathrm{run}\}$, let P_t^χ be the statement that $F_{x,t,\pi}^\chi$ is satisfiable for some head schedule π. By Lemma 5.1(b), P_t^{run} holds if and only if some computation of $\mathcal{M}$ on x has not halted after $\bar{t}$ steps, and P_t^{acc} holds if and only if some computation is in an accepting configuration after $\bar{t}$ steps.

Testing P_t^χ. Enumerate the head schedules π and apply Corollary 4.8 to each $F_{x,t,\pi}^\chi$, whose number of variables, scopes and tables are supplied by the transducer of Lemma 5.1; it has $N \geq t$ variables and arity at most $C \log_2 t \leq C \log_2 N$. This uses space $O(t/\log t)$ for π, $O(\kappa(Ct)) = O(\kappa(t))$ for Corollary 4.8, and $O(\log t)$ for the transducer, so $O(\kappa(t))$ in all.

Doubling. For $t = t_0, 2t_0, 4t_0, \ldots$, where $t_0 = \max(n, 4)$, test P_t^{run}; at the first t for which it fails, accept if P_t^{acc} holds and reject otherwise. At that point every computation of $\mathcal{M}$ on x has halted after $\bar{t}$ steps, and a halted computation stays halted, so P_t^{acc} holds if and only if some computation accepts, that is, if and only if $\mathcal{M}$ accepts x. Since $\bar{t} \geq t$ and every computation halts within $T(n)$ steps, the loop stops at the latest when $t \geq T(n)$, so the final t is at most $2\max(T^*(n), 4)$. The stages reuse the same space, so the total space is $O(\kappa(T^*))$, using $\kappa(2T) = O(\kappa(T))$. For the complement, the deterministic machine gives the opposite answer. □

Proof of Theorem 2. This is Theorem 6.1. □

Let $\delta(x) = (\log\log x)^2/\sqrt{\log x}$, so that $\kappa(x) = x\,\delta(x)$ and $\rho(x) = x/\delta(x)$. The function δ is eventually decreasing and tends to 0, and κ is eventually increasing. Hence, for all sufficiently large S and T,

$$\frac{\kappa(T)}{S} \leq \delta(S) + \frac{T}{\rho(S)}, \tag{6}$$

because $\kappa(T) \leq \kappa(S) = S\,\delta(S)$ if $T \leq S$, and $\kappa(T) = T\,\delta(T) \leq T\,\delta(S) = S\,T/\rho(S)$ if $T > S$.

Theorem 6.2. *Let $S(n) \geq n$ be space-constructible, and let $\mathcal{M}$ be a nondeterministic machine that decides L_S or its complement, with running time $T(n)$. Then $T(n) \geq c\,\rho(S(n))$ for some $c > 0$ and all large n. In particular $L_S \notin \mathrm{NTIME}(T) \cup \mathrm{coNTIME}(T)$ for every $T = o(\rho \circ S)$, and $\mathrm{NTIME}(S) \subsetneq \mathrm{DSPACE}(S)$ and $\mathrm{NTIME}(S) \subsetneq \mathrm{NSPACE}(S)$. Moreover $\mathrm{NTIME}(n) \subsetneq \mathrm{DSPACE}(S')$ for every space-constructible S' with $\kappa(n) = o(S'(n))$, for example $S'(n) = \lfloor n/(\log n)^{1/2-\varepsilon} \rfloor$ with $\varepsilon > 0$ rational.*

Proof. Otherwise $T(n)/\rho(S(n)) \to 0$ along a sequence of lengths n. Along it, $T^*(n) = \max(T(n), n)$ also satisfies $T^*(n)/\rho(S(n)) \to 0$, because $n/\rho(S(n)) \leq \delta(S(n)) \to 0$; and $T^*(n) \geq n$ and $S(n) \geq n$ tend to infinity. By (6), $\kappa(T^*(n))/S(n) \leq \delta(S(n)) + T^*(n)/\rho(S(n)) \to 0$ along the sequence. By Theorem 6.1, L_S is decided in space $O(\kappa(T^*(n)))$, which contradicts Lemma 2.1. Next, Theorem 6.1 gives $\mathrm{NTIME}(S) \subseteq \mathrm{DSPACE}(\kappa(S)) \subseteq \mathrm{DSPACE}(S)$, and $L_S \notin \mathrm{NTIME}(S)$ because $S = o(\rho \circ S)$; since $\mathrm{DSPACE}(S) \subseteq \mathrm{NSPACE}(S)$, both inclusions are strict. Finally, $\mathrm{NTIME}(n) \subseteq \mathrm{DSPACE}(\kappa(n)) \subseteq \mathrm{DSPACE}(S')$, and $L_{S'} \in \mathrm{DSPACE}(S')$ is not in $\mathrm{DSPACE}(\kappa(n))$ by Lemma 2.1, so it is not in $\mathrm{NTIME}(n)$. □

Proof of Theorem 1. Take $S(n) = n$ in Theorem 6.2 and $L = L_n$. □

Corollary 6.3 (Toward the question of Hopcroft, Paul and Valiant)**.** *For every $t(n) \geq n$,*

$$\mathrm{NTIME}\Big(\frac{t\sqrt{\log t}}{(\log\log t)^2}\Big) \subseteq \mathrm{DSPACE}(t).$$

Proof. Let $T = t\sqrt{\log t}/(\log\log t)^2$. For large t we have $t \leq T \leq t\sqrt{\log t}$, so $\log t \leq \log T \leq 2\log t$ and $\log\log T \leq \log\log t + 1$. Hence

$$\kappa(T) = \frac{T(\log\log T)^2}{\sqrt{\log T}} \leq t\Big(\frac{\log\log t + 1}{\log\log t}\Big)^2 = O(t),$$

and $\kappa(\max(T, n)) = O(t)$ because $\kappa(n) \leq \kappa(t) \leq t$ for large n. Theorem 6.1 gives $\mathrm{NTIME}(T) \subseteq \mathrm{DSPACE}(O(t)) = \mathrm{DSPACE}(t)$. □

6.1 Probabilistic and parity time

A probabilistic machine tosses a fair coin at every step and ignores it when only one transition is applicable; if all its computations on x halt within $\bar{t}$ steps, it accepts x with probability $2^{-\bar{t}}$ times the number of coin sequences in $\{0,1\}^{\bar{t}}$ that lead to acceptance. $\mathrm{BPTIME}(T)$ and $\mathrm{PPTIME}(T)$ are defined by bounded two-sided error and by acceptance probability greater than $1/2$, and $\oplus\mathrm{TIME}(T)$ consists of the languages of inputs on which a nondeterministic machine running in time T has an odd number of accepting computations.

We use the following classical fact.

Fact 6.4 (Chinese remaindering in logarithmic space [4, 6])**.** *Let a and b be constants. There is a deterministic transducer that, given distinct primes $p_1, \ldots, p_L$ with $L \leq u^a$ and $p_i \leq u^b$, and the residues modulo p_i of an integer $0 \leq X < \prod_i p_i$, on a read-only input tape, writes the binary representation of X on a write-only output tape in space $O(\log u)$.*

Corollary 6.5. *For $T(n) \geq n$, the classes $\mathrm{BPTIME}(T)$, $\mathrm{PPTIME}(T)$ and $\oplus\mathrm{TIME}(T)$ are contained in $\mathrm{DSPACE}(\kappa(T))$. None of them contains the language of Theorem 1 when $T = o(\rho)$.*

Proof. Find the final t of the doubling loop of Theorem 6.1; then every computation halts within $\bar{t}$ steps. By Lemma 5.1, the numbers α and ϱ of accepting and rejecting choice strings in $\{0,1\}^{\bar{t}}$ (for $\oplus$TIME: of accepting computations, using the requirement on choice bits) are sums $\sum_\pi \#F_\pi^\chi$, and both are at most $2^{\bar{t}} < 4^t$. If t is below a constant threshold, compute α and ϱ by brute force. Otherwise let $p_1, \dots, p_t$ be the t smallest primes larger than $4C^2t^2$; they lie in $(4C^2t^2, 8C^2t^2]$, by the prime number theorem, so they are admissible in Theorem 4.6 for every F_π^χ, which has between t and Ct variables; and their product exceeds 4^t. The residues of α and ϱ modulo each p_i are therefore computed by summing the outputs of Theorem 4.6 over all head schedules, and the bits of α and ϱ are obtained from these residues by the transducer of Fact 6.4, with $u = t$, recomputing each residue when it is requested. Accept if $\alpha > \varrho$, comparing bit by bit from the top (for BPTIME and PPTIME), or if α is odd (for $\oplus$TIME). The last statement follows as in the proof of Theorem 6.2. □

7 Limits and open questions

The proofs have not been independently verified. The part that most needs checking is Section 4, in particular Lemma 4.5 and the space accounting of Theorem 4.6, which Appendix B spells out. Next comes Proposition 3.3, which Kintali states for fixed parameters and we prove for parameters that grow with N. The checks of Appendix C test only the finite algebra, at toy sizes.

The exponent $\frac{1}{2}$ in κ comes from the construction. We do not see how to reduce the space below $N/\sqrt{\log N}$, up to lower-order factors, by retuning the construction while enumerating marked supports as we do. In our analysis the number of measurements carries the factor $e^{-R^2/(8Q)}$, so R must exceed $\sqrt{Q}$ for it to be small. The marked supports then have up to a fraction $3R/Q \gtrsim 1/\sqrt{Q}$ of the variables, and since $D = r^{2Q} \le N$ forces $Q = O(\log N)$, this fraction is of order at least $1/\sqrt{\log N}$. A counting algorithm with space $N/(\log N)^\alpha$ and the same interface would give the separations up to time $n(\log n)^{\alpha - o(1)}$. Corollary 6.3 stops at $t\sqrt{\log t}$, up to $\log\log$ factors, so the question of Hopcroft, Paul and Valiant, whether $\mathrm{NTIME}(t \log t) \subseteq \mathrm{NSPACE}(t)$, remains open.

Of the two factors $\log\log N$ in $\kappa(N)$, one comes from writing the supports, which costs k bits per element, and the other from the ratio R/Q. The time bound of [13, Theorem 1.1], for formulas with a linear number of clauses, has no such factor, but the tensor-network recursion behind it [13, Theorem D.1] works with a 2-adic precision larger than the number of wires. At a call on a network with W wires that retains u valuation levels, the precision at the interpolation nodes it uses is at least $24W\,2^u/u$ [13, Lemma D.5 and the proof of (D.11)], so a single intermediate value already exceeds sublinear space. A small-space version of that recursion would remove $(\log\log)^2$ from all our bounds and give $\mathrm{NTIME}(t\sqrt{\log t}) \subseteq \mathrm{DSPACE}(t)$.

Theorem 1 does not relativize. Let BSA be the set of strings $\langle \mathcal{M} \rangle \# x \# 1^b$ such that the deterministic machine $\mathcal{M}$, which has one binary work tape, accepts x without using more than b work cells. Then $\mathsf{BSA} \in \mathrm{DSPACE}(n)$, and every language in $\mathrm{DSPACE}(n)$ reduces to it by a map computable in linear time with linear output length. With BSA as an oracle, written on a query tape that counts toward space, $\mathrm{DSPACE}^{\mathsf{BSA}}(n) = \mathrm{DSPACE}(n) \subseteq \mathrm{DTIME}^{\mathsf{BSA}}(n)$, so $\mathrm{DTIME}^{\mathsf{BSA}}(n) = \mathrm{NTIME}^{\mathsf{BSA}}(n) = \mathrm{DSPACE}^{\mathsf{BSA}}(n)$. The same oracle shows that the separations of [9] and [14] do not relativize. Like them, our proof uses the locality of multitape computations, through Lemma 5.1.

Strictness at every polynomial scale does not separate NP from PSPACE, since the witness

language depends on the exponent, and nothing here bears on DSPACE(n) versus NSPACE(n). The simulation needs acceptance to be a function of a single count, which fails under unrestricted alternation, including Σ_2-time and Arthur–Merlin protocols. It would also need the computations to be described by a bounded-occurrence constraint system with $O(T \operatorname{poly}\log\log T)$ variables, and we know no such description for machines with random access to their memory.

A Kintali's construction and the proof of Proposition 3.3

This appendix gives Kintali's construction [11, §3] with all definitions, proves Proposition 3.3 along the lines of [11, §3], for constraints of any arity and for all parameters, and proves Lemma 4.5. Fix integers $r \geq 2$, $Q \geq 1$ and $R \geq 2$.

The matrix. Let $\mathcal{B} = ([r] \times [r])^Q$, so $D = |\mathcal{B}| = r^{2Q}$, and write $u \in \mathcal{B}$ as $((a_u(t), d_u(t)))_{t\in[Q]}$. For $t \in [Q]$ let Π_t be the permutation matrix on $\mathbb{Q}^{\mathcal{B}}$ that swaps the two entries of pair t, and put $V = \prod_t \Pi_t$ and $W = \frac{1}{2}\sum_t (I - \Pi_t)$. For one pair, an orthogonal eigenbasis of the swap consists of the r vectors e_{aa}, the $\binom{r}{2}$ symmetric vectors $e_{ab} + e_{ba}$ and the $\binom{r}{2}$ antisymmetric vectors $e_{ab} - e_{ba}$, for $a < b$; tensor products of one-pair basis vectors form an orthogonal basis Ψ of $\mathbb{Q}^{\mathcal{B}}$, with $|\Psi| = D$ and entries in $\{0, \pm 1\}$. If $\psi \in \Psi$ has $j(\psi)$ antisymmetric factors then $W\psi = j(\psi)\psi$ and $V\psi = (-1)^{j(\psi)}\psi$, and $\langle\psi, \psi\rangle$ is 2 to the number of its factors that are not of the form e_{aa}.

Let $\mu = Q(r-1)/(2r)$, $b_0 = \lfloor \mu - (R-1)/2 \rfloor$ and $I_0 = \{b_0, \dots, b_0 + R - 1\}$, and let

$$f(y) = (-1)^{b_0} \sum_{a=0}^{R-1} (-2)^a \binom{y - b_0}{a}.$$

This polynomial has degree at most $R-1$, and $f(b_0 + i) = (-1)^{b_0} \sum_{a\leq i} \binom{i}{a}(-2)^a = (-1)^{b_0+i}$ for $0 \leq i < R$; so it is the interpolation polynomial of the signs $(-1)^j$ on I_0. Let $A = Vf(W)$, let $\Psi_{\text{bad}} = \{\psi \in \Psi : j(\psi) \notin I_0\}$, and

$$\beta_{u,\psi} = \frac{(1 - (-1)^{j(\psi)} f(j(\psi)))\psi_u}{\langle\psi, \psi\rangle}.$$

Since $A\psi = (-1)^{j(\psi)} f(j(\psi))\psi$, which is ψ when $j(\psi) \in I_0$, the expansion of $I - A$ in the orthogonal basis Ψ gives

$$A[u, w] = \mathbf{1}[u = w] - \sum_{\psi\in\Psi_{\text{bad}}} \beta_{u,\psi}\psi_w. \tag{7}$$

For $u, w \in \mathcal{B}$ let $E_{u,w} = \{t : a_w(t) \neq d_u(t)\}$ and $\phi_{u,w}(T) = \prod_{t\in E_{u,w}}(1 - T/t)$, and define

$$P_u(T; \zeta) = \sum_{\psi\in\Psi_{\text{bad}}} \beta_{u,\psi}\langle\psi, \zeta\rangle + \sum_{w\in\mathcal{B}} A[u, w]\, \phi_{u,w}(T)\, \zeta_w.$$

The denominators occurring here are powers of two and the nodes $t \in [Q]$ (the polynomial f takes integer values at integers), so all these numbers are p-integral for every prime $p > \max(Q, R)$.

Lemma A.1 ([11, Lemma 3.2])**.** $|\Psi_{\text{bad}}| \leq 2D\, e^{-R^2/(8Q)}$.

Proof. If ψ is chosen uniformly from Ψ, its Q factors are independent and uniform among the r^2 one-pair basis vectors, of which $\binom{r}{2}$ are antisymmetric. So $j(\psi)$ is a sum of Q independent

Bernoulli variables with mean $(r-1)/(2r)$, and its mean is μ. By the choice of b_0, every integer $j < b_0$ satisfies $\mu - j \geq (R+1)/2$, and every integer $j \geq b_0 + R$ satisfies $j - \mu > (R-1)/2$; as $R \geq 2$, every $j \notin I_0$ thus has $|j-\mu| \geq R/4$. Hoeffding's inequality [7] gives

$$\frac{|\Psi_{\mathrm{bad}}|}{D} = \Pr[j(\psi) \notin I_0] \leq \Pr[|j(\psi)-\mu| \geq R/4] \leq 2e^{-2(R/4)^2/Q} = 2e^{-R^2/(8Q)}. \qquad \square$$

Lemma A.2 ([11, Lemma 3.3])**.** *If $A[u,w] \neq 0$, then $|E_{u,w}| \leq R-1$.*

Proof. The matrices Π_t commute and satisfy $\Pi_t^2 = I$; for $S \subseteq [Q]$ put $\Pi_S = \prod_{t \in S} \Pi_t$. Expanding the product shows that W^i is a linear combination of matrices $\Pi_{S'}$ with $|S'| \leq i$. Since $\deg f \leq R-1$, $f(W)$ is a linear combination of matrices $\Pi_{S'}$ with $|S'| \leq R-1$, and $A = Vf(W)$ is a linear combination of the matrices $V\Pi_{S'} = \Pi_{[Q]\setminus S'}$. Now $\Pi_S[u,w] \neq 0$ means that u arises from w by swapping the two entries of every pair $t \in S$; then $d_u(t) = a_w(t)$ for $t \in S$, so $E_{u,w} \subseteq [Q] \setminus S$. Hence $A[u,w] \neq 0$ implies $E_{u,w} \subseteq S'$ for some $|S'| \leq R-1$. $\square$

Labels, copies and forms. Given group schedules a and d, incidence (e,v) has label $u(e,v)$, and c_u incidences have label u. With $h = \max_u c_u$ copies of $\mathcal{B}$, place every incidence at position $u(e,v)$ in a copy $j(e,v) \in [h]$ not used by another incidence with the same label; we use the copy given by the rank of the incidence, in lexicographic order, among the incidences with the same label. Let $O_j \subseteq \mathcal{B}$ be the occupied positions of copy j, $\nu_j(w)$ the variable stored at $w \in O_j$, and $\zeta^{(j)}(X)$ the vector with $\zeta^{(j)}_w = X_{\nu_j(w)}$ for $w \in O_j$ and 0 otherwise. Define

$$L_{e,v}(T;X) = P_{u(e,v)}(T; \zeta^{(j(e,v))}(X)), \qquad S_{t,g} = \{v : a_v(t) = g\}, \qquad \sigma_t(e) = d_e(t),$$

and the measurements $m_{t,j,\psi,g}(X) = \sum_{w \in O_j,\ a_w(t)=g} \psi_w X_{\nu_j(w)}$ for $j \in [h]$, $\psi \in \Psi_{\mathrm{bad}}$ and $g \in [r]$.

Proof of Proposition 3.3. Fix an incidence (e,v) and put $u = u(e,v)$ and $j = j(e,v)$. The incidence is stored at position u of copy j, so $\zeta^{(j)}_u(X) = X_v$.

Value at 0 and degree. Since $\phi_{u,w}(0) = 1$, (7) gives, for every vector ζ,

$$P_u(0;\zeta) = \sum_{\psi \in \Psi_{\mathrm{bad}}} \beta_{u,\psi} \langle \psi, \zeta \rangle + \sum_{w \in \mathcal{B}} \Big(\mathbf{1}[u=w] - \sum_{\psi \in \Psi_{\mathrm{bad}}} \beta_{u,\psi} \psi_w \Big) \zeta_w = \zeta_u,$$

so $L_{e,v}(0;X) = X_v$. The measurement part of P_u does not depend on T, and $\phi_{u,w}$ has degree $|E_{u,w}| \leq R-1$ whenever $A[u,w] \neq 0$, by Lemma A.2; so $\deg_T L_{e,v} \leq R-1$.

The data at a node. (D1) and (D2) hold by definition and (3). Fix a node t. By the definitions,

$$L_{e,v}(t;X) = \sum_{\psi \in \Psi_{\mathrm{bad}}} \beta_{u,\psi} \langle \psi, \zeta^{(j)}(X) \rangle + \sum_{w \in O_j} A[u,w]\, \phi_{u,w}(t)\, X_{\nu_j(w)}.$$

Every $w \in O_j$ is the label of an incidence $(e', \nu_j(w))$, so $a_w = a_{\nu_j(w)}$, that is, $\nu_j(w) \in S_{t,a_w(t)}$. Hence $\langle \psi, \zeta^{(j)}(X) \rangle = \sum_g m_{t,j,\psi,g}(X)$, where $m_{t,j,\psi,g}$ is supported on $S_{t,g}$ and has integer coefficients with ℓ_1-norm at most $|O_j| \leq D$, because ψ has entries in $\{0, \pm 1\}$; this is (D3). In the second sum $\phi_{u,w}(t) = 0$ unless $a_w(t) = d_u(t) = d_e(t) = \sigma_t(e)$, and then $\nu_j(w) \in S_{t,\sigma_t(e)}$. This proves (D4), in the form

$$L_{e,v}(t;X) = \sum_{\psi \in \Psi_{\mathrm{bad}}} \beta_{u,\psi} \sum_{g=1}^{r} m_{t,j,\psi,g}(X) + \sum_{w \in O_j,\ a_w(t)=d_e(t)} A[u,w]\, \phi_{u,w}(t)\, X_{\nu_j(w)}. \tag{8}$$

The measurements at node t are indexed by $(g, j, \psi) \in [r] \times [h] \times \Psi_{\mathrm{bad}}$, and $h = \max_u c_u \le 2M/D$ by (3). With Lemma A.1, there are at most $rh|\Psi_{\mathrm{bad}}| \le 4Mr\, e^{-R^2/(8Q)}$ of them. □

Proof of Lemma 4.5. The seed is stored once, in $O(\log^2 N)$ bits. Each item below computes its quantity from indices of $O(\log N)$ bits (incidences, nodes, labels, copies, groups, elements of Ψ) and the seed, with $O(\log N)$ bits of workspace besides the workspace of the earlier items that it calls. Calls are nested to a depth bounded by a constant, so every quantity is computed in space $O(\log^2 N)$, and nothing else is stored.

(P1) *Constraints and incidences.* Number the constraints of F as the transducer does, followed by the dummy constraints of Section 3 in the order of their variables; whether a variable lies in no scope is checked by scanning the scopes. This gives the index, scope and table of every constraint, the numbers $|\mathcal{F}|$ and M, and the incidences in lexicographic order.

(P2) *Group schedules and labels.* The values $a_v(t)$ and $d_e(t)$ are k bits of the affine maps of Section 4.4, computed from the seed and the binary vector of v or e; the label $u(e, v)$ has $2Qk \le 2\log_2 N$ bits. Membership $v \in S_{t,g}$ is a comparison; the position of v in $S_{t,g}$ is the number of $v' < v$ with $a_{v'}(t) = g$, and $n_{t,g}$ is computed by one scan over the variables.

(P3) *Counts and copies.* The count c_u is computed by one scan over the incidences, and $h = \max_u c_u$ by a loop over the $D \le N^{1/8}$ labels. The copy $j(e, v)$ is one plus the number of lexicographically earlier incidences with label $u(e, v)$. The variable $\nu_j(w)$, or the fact that $w \notin O_j$, is found by a scan over the incidences with label w, computing their copies.

(P4) *Basis vectors.* An element $\psi \in \Psi$ is given by its Q one-pair factors, in $O(Q \log r) = O(\log N)$ bits. The entry ψ_w is a product of Q one-pair entries in $\{0, \pm 1\}$; $j(\psi)$ and $\langle \psi, \psi \rangle$ are read off from the factors.

(P5) *Spectral coefficients.* The value $f(j) \bmod p$ is a sum of R terms of the explicit formula, each binomial coefficient computed modulo p by the multiplicative formula, as $a < R < p$. Then $\beta_{u,\psi} \bmod p$ follows from its definition, and $A[u, w] \bmod p$ from (7), by a loop over the D elements of Ψ that skips those with $j(\psi) \in I_0$.

(P6) *Deformation factors.* The value $\phi_{u,w}(\tau) \bmod p$ at a point $\tau \in \mathbb{F}_p$ is a product over the nodes in $E_{u,w}$.

(P7) *Measurements.* The measurements at node t are indexed by $(g, j, \psi) \in [r] \times [h] \times \Psi_{\mathrm{bad}}$, in lexicographic order, so that the values y_g of the measurements of each group are consecutive; the tuple y stores the values in this order, each plus D in a field of $\lceil \log_2(2D + 1) \rceil$ bits, and the position of the field of (g, j, ψ) is computed by counting the earlier indices. The value $m_{t,j,\psi,g}(x)$ is the sum of $\psi_w x_{\nu_j(w)}$ over the $w \in O_j$ with $a_w(t) = g$, an integer in $\{-D, \dots, D\}$ accumulated exactly by a loop over the pairs $(w, \nu_j(w))$, $w \in O_j$, which (P3) generates by a scan of the incidences; the bit $x_{\nu_j(w)}$ is read from the oracle for x (in Theorem 4.6, from the stored assignment x_g, at the position of $\nu_j(w)$ in $S_{t,g}$).

Now (a): by (8), the coefficient of the measurement $m_{t,j',\psi,g}$ in $L_{e,v}(t; X)$ is $\beta_{u,\psi}$ if $j' = j$ and 0 otherwise, and the coefficient of a variable X_i of the group $S_{t,\sigma_t(e)}$ is the sum of $A[u, w]\, \phi_{u,w}(t)$ over the $w \in O_j$ with $a_w(t) = d_e(t)$ and $\nu_j(w) = i$, where $u = u(e, v)$ and $j = j(e, v)$; items (P3)–(P6) compute them. Part (b) is item (P7). For (c), the measurement part of P_u does not depend on T, so

$$b^U_{e,v}(z) = \sum_{w \in O_j} A[u, w](\phi_{u,w}(t + z) - \phi_{u,w}(t))\mathbf{1}[\nu_j(w) \in U], \qquad u = u(e, v),\ j = j(e, v),$$

which is computed by a loop over the pairs $(w, \nu_j(w))$, $w \in O_j$, generated as in (P3), that calls items (P5), (P6) and the membership oracle for U. □

B The counting algorithm in pseudocode

This appendix writes out the algorithm of Theorem 4.6, so that the call chain

$$\texttt{MOD_COUNT} \;\to\; \texttt{GROUP_SUM} \;\to\; \texttt{CONSTRAINT_VALUE} \;\to\; \texttt{NODEVALUE}, \texttt{CORRECTION}$$

and the lifetime of every buffer can be checked line by line. A direct transcription into Python, with the same routines, agrees with brute-force counts on toy systems (Appendix C, item 7). The parameters are those of Section 4.1, and the notation is that of Section 4 and Appendix A.

Input. The transducer gives N, the constraints, their scopes and their tables modulo p (Section 2). Constraints with empty scope are constant factors: `MOD_COUNT` multiplies their values first and ignores them afterwards. The other constraints, followed by the dummy constraints of Section 3, are numbered $1, \dots, |\mathcal{F}|$ as in (P1); $\mathrm{SCOPE}(e, i)$ is the ith element of I_e in increasing order, and $\mathrm{TABLE}(e, a) = \gamma_e(a) \bmod p$. The incidences are the pairs (e, i) with $i \le |I_e|$, in lexicographic order. The seed is stored once and defines $a_v(t)$, $d_e(t)$ and $\mathrm{LABEL}(e, v) = u(e, v)$ (Section 4.4).

Buffers. Three objects are large. Each occupies a fixed region of a work tape and is written only by the routines named below.

- ETA holds $\mathrm{code}(\eta)$ for the current marked support, and its length in a separate counter of $O(\log N)$ bits. Its writer is the loop over supports in `MOD_COUNT`, which runs through all binary strings of length at most Λ by incrementing ETA in place.
- Y holds the current tuple y: $n_y = rh|\Psi_{\mathrm{bad}}|$ fields of $w_y = \lceil \log_2(2D+1) \rceil$ bits, in the order of (P7); the field of the measurement (g, j, ψ) is the field number $\big((g-1)h + j - 1\big)|\Psi_{\mathrm{bad}}| + \mathrm{rank}(\psi)$, where $\mathrm{rank}(\psi)$ is the position of ψ in Ψ_{bad}, and it holds the value plus D. It is written only by `MOD_COUNT`, which sets it to zeros for each support, and by `NEXT_Y`, which advances it.
- X holds the assignment x_g to the current group $S_{t,g}$, the bit of v at the position of v in $S_{t,g}$. Its writer is `GROUP_SUM`, and at most one group assignment exists at any time.

All other routines receive these buffers as read-only arguments, which a multitape machine realizes by moving a head to an offset kept in $O(\log N)$ bits and back. No routine copies more than $O(\log N)$ bits of a buffer: `YGET`, `ETAGET` and `XGET` return a single field or bit.

The main routines. Arithmetic is modulo p, except in `MEASVAL`, which computes an integer in $\{-D, \dots, D\}$ exactly.

```
function MOD_COUNT( )                                          ▷ returns #F mod p
    c_∅ ← ∏_{e: I_e=∅} γ_e(∅); if c_∅ = 0 return 0
    if N is below the threshold of Lemma 4.1 return #F mod p by brute force
    compute the parameters; seed ← FIND_SEED(); h ← MAX_COPIES(); n_y ← rh|Ψ_bad|
    acc ← 0
    for t = 1, ..., Q and z = 1, ..., p − 1 do
        ω ← WEIGHT(t, z); sum ← 0                              ▷ the recursion of COEFF ends here
        for every binary string ETA of length at most Λ do
            if VALID_ETA(ETA) then
                Y ← 0^{n_y w_y}                                ▷ every field encodes the value −D
                repeat
                    prod ← (−1)^{NEGCOUNT(ETA)}
                    for g = 1, ..., r do
```

$prod \leftarrow prod \cdot \texttt{GROUP_SUM}(t, z, g, \mathsf{ETA}, \mathsf{Y})$
$sum \leftarrow sum + prod$
until not $\texttt{NEXT_Y}(\mathsf{Y})$
$acc \leftarrow acc + \omega \cdot sum$
return $c_\emptyset\, \Omega(0)\, acc$ ▷ $\Omega(0) = \prod_{t'=1}^{Q} (-t')^J$

function $\texttt{GROUP_SUM}(t, z, g, \mathsf{ETA}, \mathsf{Y})$ ▷ the sum over x_g in Proposition 4.3
$\mathsf{X} \leftarrow 0^{n_{t,g}}$; $tot \leftarrow 0$ ▷ the only writer of X
loop
if $\texttt{VALID_GROUP}(t, g, \mathsf{ETA}, \mathsf{Y}, \mathsf{X})$ **then**
$prod \leftarrow 1$
for $e = 1, \ldots, |\mathcal{F}|$ with $d_e(t) = g$ **do**
$prod \leftarrow prod \cdot \texttt{CONSTRAINT_VALUE}(e, t, z, g, \mathsf{ETA}, \mathsf{Y}, \mathsf{X})$
$tot \leftarrow tot + prod$
if $\mathsf{X} = 1^{n_{t,g}}$ **return** tot
increment X in binary, in place

function $\texttt{VALID_GROUP}(t, g, \mathsf{ETA}, \mathsf{Y}, \mathsf{X})$ ▷ the conditions on x_g
for $v \in [N]$ with $a_v(t) = g$ **do**
if $\texttt{ETAGET}(\mathsf{ETA}, v) \neq 0$ and $\texttt{XGET}(\mathsf{X}, t, g, v) = 0$ **return** false ▷ $x_v = 1$ on $E(\eta)$
for $j \in [h]$ and $\psi \in \Psi_{\text{bad}}$ **do**
if $\texttt{MEASVAL}(t, g, j, \psi, \mathsf{X}) \neq \texttt{YGET}(\mathsf{Y}, g, j, \psi)$ **return** false ▷ $C_g x_g = y_g$
return true

function $\texttt{CONSTRAINT_VALUE}(e, t, z, g, \mathsf{ETA}, \mathsf{Y}, \mathsf{X})$ ▷ $\Phi_e(\xi)$ with $\xi_v = \lambda_{e,v}(x_g; y) + b_{e,v}^{U(\eta)}(z)$
$val \leftarrow 0$
for $a \in \{0,1\}^{|I_e|}$ **do** ▷ a counter of $|I_e| \le c_1 \log_2 N$ bits
$prod \leftarrow \text{TABLE}(e, a)$
for $i = 1, \ldots, |I_e|$ **do**
$\xi \leftarrow \texttt{NODEVALUE}(e, i, t, g, \mathsf{X}, \mathsf{Y}) + \texttt{CORRECTION}(e, i, t, z, \mathsf{ETA})$
$prod \leftarrow prod \cdot \big(a_i \xi + (1 - a_i)(1 - \xi)\big)$
$val \leftarrow val + prod$
return val

function $\texttt{NODEVALUE}(e, i, t, g, \mathsf{X}, \mathsf{Y})$ ▷ $\lambda_{e,v}(x_g; y)$ by (8); here $g = d_e(t)$
$v \leftarrow \text{SCOPE}(e, i)$; $u \leftarrow \text{LABEL}(e, v)$; $j \leftarrow \texttt{COPYRANK}(e, i)$; $\lambda \leftarrow 0$
for $\psi \in \Psi_{\text{bad}}$ **do**
$\lambda \leftarrow \lambda + \texttt{BETA}(u, \psi) \cdot \sum_{g'=1}^{r} \texttt{YGET}(\mathsf{Y}, g', j, \psi)$
for $(w, v') \in \texttt{OCCUPIED}(j)$ with $a_{v'}(t) = g$ **do**
$\lambda \leftarrow \lambda + \texttt{AENTRY}(u, w) \cdot \texttt{PHILOC}(u, w, t) \cdot \texttt{XGET}(\mathsf{X}, t, g, v')$
return λ

function $\texttt{CORRECTION}(e, i, t, z, \mathsf{ETA})$ ▷ $b_{e,v}^{U(\eta)}(z)$, as in the proof of Lemma 4.5(c)
$v \leftarrow \text{SCOPE}(e, i)$; $u \leftarrow \text{LABEL}(e, v)$; $j \leftarrow \texttt{COPYRANK}(e, i)$; $b \leftarrow 0$
for $(w, v') \in \texttt{OCCUPIED}(j)$ with $\texttt{ETAGET}(\mathsf{ETA}, v') = +$ **do**
$b \leftarrow b + \texttt{AENTRY}(u, w) \cdot \big(\texttt{PHILOC}(u, w, t + z) - \texttt{PHILOC}(u, w, t)\big)$
return b

function $\texttt{MEASVAL}(t, g, j, \psi, \mathsf{X})$ ▷ $m_{t,j,\psi,g}(x)$, an integer in $\{-D, \ldots, D\}$
$m \leftarrow 0$

for $(w, v) \in$ `OCCUPIED`(j) with $a_v(t) = g$ **do**
  $m \leftarrow m + \psi_w \cdot$ `XGET`(X, t, g, v)
return m

function `NEXT_Y`(Y) ▷ increment in base $2D+1$, the last field fastest
for $o = n_y, n_y - 1, \ldots, 1$ **do**
  if field o of Y is less than $2D$: increase it by 1, set fields $o+1, \ldots, n_y$ to 0, **return** true
return false

function `WEIGHT`(t, z) ▷ $z^{p-J} G_t^{<J}(z)$
return $z^{p-J} \sum_{d=0}^{J-1}$ `COEFF`$(1, Q, d, t)\, z^d$ ▷ one term at a time

function `COEFF`(lo, hi, d, t) ▷ $[Z^d]$ of the product of the factors $lo, \ldots, hi$ of G_t
if $lo = hi$ **return** `LEAF_COEFF`(lo, t, d)
$mid \leftarrow \lfloor (lo + hi)/2 \rfloor$
return $\sum_{a=0}^{d}$ `COEFF`$(lo, mid, a, t) \cdot$ `COEFF`$(mid + 1, hi, d - a, t)$ ▷ one term at a time

The remaining routines. Each of them keeps $O(1)$ scalars of $O(\log N)$ bits (indices, labels, elements of Ψ, field elements) besides its arguments, and the loops in the column on the right are scans of the incidences, the variables, Ψ or the labels.

routine	value, and how it is computed
`YGET`(Y, g, j, ψ)	the field of (g, j, ψ) in Y, minus D; rank(ψ) (`BADRANK`) by a scan of Ψ
`XGET`(X, t, g, v)	the bit of X at the position of v in $S_{t,g}$, which is $\lvert\{v' < v : a_{v'}(t) = g\}\rvert$, counting from 0 as in (P2)
`VALID_ETA`(ETA)	whether $\mathsf{ETA} =$ code(η) for a marked support with $\lvert\eta\rvert \le q$, by the test of Section 4.5
`ETAGET`(ETA, v)	$\eta_v \in \{0, +, -\}$, by reading ETA from the left
`NEGCOUNT`(ETA)	$\lvert\eta^-\rvert$, by reading ETA from the left
`COPYRANK`(e, i)	the copy $j(e, v)$ of the incidence (e, i): the number of incidences $(e', i') \le (e, i)$ with the same label (P3)
`OCCUPIED`(j)	generates the pairs $(w, \nu_j(w))$, $w \in O_j$: the pairs $(u(e, v), v)$ for the incidences (e, i) with `COPYRANK`$(e, i) = j$
`MAX_COPIES()`	$h = \max_u c_u$, by a loop over the D labels
`FVALUE`(i)	$f(i) = (-1)^{b_0} \sum_{a<R} \tau_a$, where $\tau_0 = 1$ and $\tau_{a+1} = \tau_a \cdot (-2)(i - b_0 - a)/(a+1)$ (P5)
`BETA`(u, ψ)	$\beta_{u,\psi} = \big(1 - (-1)^{j(\psi)}$`FVALUE`$(j(\psi))\big)\psi_u / \langle\psi, \psi\rangle$ (P4, P5)
`AENTRY`(u, w)	$A[u, w] = \mathbf{1}[u = w] - \sum_{\psi \in \Psi_{\text{bad}}}$ `BETA`$(u, \psi)\psi_w$, by (7)
`PHILOC`(u, w, τ)	$\phi_{u,w}(\tau) = \prod(1 - \tau/t')$ over the nodes t' with $a_w(t') \ne d_u(t')$ (P6)
`LEAF_COEFF`(a, t, d)	$[Z^d](c + Z)^{-m} = (-1)^d \binom{m+d-1}{d} c^{-m-d}$, where $(c, m) = (t, 1)$ if $a = t$ and $(c, m) = (t - a, J)$ otherwise
`FIND_SEED()`	the first seed, in lexicographic order, whose group schedules satisfy (3); each c_u and each $n_{t,g}$ is computed by a scan (Lemma 4.4)

Correctness. For fixed t, z, η and y, `GROUP_SUM`$(t, z, g, \mathsf{ETA}, \mathsf{Y})$ returns the sum over x_g in Proposition 4.3: `VALID_GROUP` tests the conditions on x_g, and `CONSTRAINT_VALUE` returns the factor of A_g^η that belongs to e, because `NODEVALUE` and `CORRECTION` return $\lambda_{e,v}(x_g; y)$ and $b_{e,v}^{U(\eta)}(z)$, by (8) and the proof of Lemma 4.5. The strings accepted by `VALID_ETA` are the codes of the marked supports with $\lvert\eta\rvert \le q$, each once (Section 4.5), and `NEXT_Y` runs through all tuples y

with entries in $\{-D, \dots, D\}$. Hence `MOD_COUNT` returns $c_\emptyset$ times the right side of Proposition 4.3 for the system without its constraints of empty scope, which is $\#F \bmod p$.

Space. At any time the following regions are live:

region	writer	lifetime	bits
`ETA` and its length	loop over supports	one pair (t, z)	$\Lambda + O(\log N)$
`Y`	`MOD_COUNT`, `NEXT_Y`	one support	$n_y w_y$
`X`	`GROUP_SUM`	one call of `GROUP_SUM`	$n_{t,g}$
seed, scalars, call stack	–	throughout	$O(\log^2 N)$

The call stack has constant depth, except during the recursion of `COEFF`, which has depth $\lceil \log_2 Q \rceil$, keeps $O(\log(pJ))$ bits per level, and ends before the loop over supports begins. Since $\Lambda \le (24k+49)N/s+1$, $n_y w_y \le 30Nk^2/s^5$ and $n_{t,g} \le 2N/s$ (Section 4.5, Lemma 4.1 and (3)), the total is $(24k+51)N/s + 30Nk^2/s^5 + O(\log^2 N) = O(\kappa(N))$. For each pair (t, z) the loops visit at most $2^{\Lambda+1}$ strings, $(2D+1)^{n_y} \le 2^{n_y w_y}$ tuples for each support and $r2^{2N/s}$ group assignments for each tuple, and every routine takes time polynomial in N, except `WEIGHT`, which takes time $J^{O(\log Q)} = 2^{O(\log N \log\log N)}$, and `FIND_SEED`, which takes time $2^{O(\log^2 N)}$; so the running time is $2^{O(\kappa(N))}$.

C Finite checks

The following exact-arithmetic checks test the finite algebra, not the asymptotic claims. The scripts accompany this draft, with a README that gives the command, the parameters and the expected output of each check.

1. *Recovery.* The projector $[z^h]P = -\sum_{z \in \mathbb{F}_p^\times} z^{p-1-h} P(z)$ for random P with $\deg P < p-1$; identity (5) for random H and Y_t with $(Q, J) \in \{(1,3), (2,4), (3,5), (4,3), (5,6)\}$, exact up to $\deg Y_t = p-2$ and failing in random trials for $\deg Y_t \in \{p-1, p\}$, so the hypothesis $\deg Y_t \le p-2$ is sharp; and Kintali's four-variable example [11, §2.4], for which (5) returns $\#F = 9$.
2. *The construction at toy sizes.* An independent re-implementation ($D \le 1024$) confirmed (8), the formula for $b^U_{e,v}$ and Lemma 3.1 with the actual measurements. End to end, on a 3-CNF with $N = 9$ variables and on systems of integer tables of arity up to 6 on $N \le 8$ variables, with the smallest J such that $QJ > M(R-1)$ and with nontrivial deformations ($\deg Y_t > J$), the untruncated Taylor sums and Lemma 4.2 returned the count modulo p, while cutting the supports one size lower gave wrong values. The master formula of Proposition 4.3, evaluated literally by its nested sums, returned $\#F \bmod p$ on three systems with $N = 6$ variables (two with tables in $\{0,1\}$, one with signed integer tables), both with q as in Section 3 and with the smallest q such that $Q(q+1) > M(R-1)$, and again failed with the supports cut one size lower. The explicit f agrees with Lagrange interpolation in 200 random cases, and (7) agrees with $A = Vf(W)$ on all 87,106 entries at six toy sizes.
3. *Appendix A.* The bound of Lemma A.1, computed exactly from the binomial law of $j(\psi)$, holds for all $r \le 8$ and $2 \le R \le Q \le 60$ (12,390 cases; the largest ratio to the bound is 0.40). With explicit matrices at five toy sizes, $A = Vf(W)$ agrees with (7), its nonzero entries satisfy Lemma A.2, and $P_u(0; \zeta) = \zeta_u$.
4. *Parameters and buffers.* The parameters (4) satisfy $R^2/(8Q) = 8k$ for $s = 2^6, \dots, 2^{11}$; the buffers of Theorem 4.6 (the code of η, the tuple y and one assignment x_g) take at most

$(24k + 51)/s + 12(2s^2k^2 + 2)/s^7$ bits per variable, which is below 1 from $s = 2^8$ on (0.95, 0.52, 0.28 for $s = 2^8, 2^9, 2^{10}$).

5. *Group schedules and supports.* The hash family of Section 4.4 is exactly pairwise uniform at small sizes, and averaging over all seeds gives exactly the value of $\mathbb{E}\Theta$ in the proof of Lemma 4.4. The strings of length at most Λ that are codes are exactly the codes of the marked supports with $|\eta| \le q$, each once (4290 marked supports, six sizes).
6. *Lemma 5.1.* On 43 random nondeterministic machines with one or two work tapes, the constraint systems, summed over all head schedules, agree with direct counts of choice strings and of computations in all 172 cases, and every variable lies in at most two scopes; an independent check that builds the systems explicitly agreed in 150 further comparisons, for the final requirements accepting, rejecting and running.
7. *Appendix B.* A direct transcription of the pseudocode into Python, with the same routines and with a guard that rejects every write to ETA, Y or X by a routine other than those named in Appendix B, returned $\#F \bmod p$, equal to a brute-force count, on five toy systems: without measurements ($\Psi_{\text{bad}} = \emptyset$) and with measurements ($|\Psi_{\text{bad}}| = 1$, up to $n_y = 4$ fields in Y), with one and with two copies of $\mathcal{B}$, with tables in $\{0, 1\}$ and with signed integer tables, and with an isolated variable, a constraint with empty scope and a group with no variables. The first condition of (3) cannot hold at these sizes, so a seed with the desired number of copies replaces `FIND_SEED`; that condition bounds sizes and is not used in the identities.

Acknowledgments

This draft was prepared through conversations with large language models (Anthropic's Claude and OpenAI's ChatGPT), which proposed and checked parts of the argument and wrote the scripts of Appendix C. Responsibility for the claims rests with the author.

References

[1] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Manuscript, 27 October 2026. https://github.com/anthropics/formal-math/tree/main/eth.

[2] R. V. Book. Comparing complexity classes. *J. Comput. System Sci.* 9(2):213–229, 1974.

[3] A. K. Chandra, D. C. Kozen, and L. J. Stockmeyer. Alternation. *J. ACM* 28(1):114–133, 1981.

[4] A. Chiu, G. Davida, and B. Litow. Division in logspace-uniform NC^1. *RAIRO Theoretical Informatics and Applications* 35(3):259–275, 2001.

[5] J. Cook and I. Mertz. Tree evaluation is in space $O(\log n \cdot \log\log n)$. In *Proc. 56th ACM STOC*, 2024.

[6] W. Hesse, E. Allender, and D. A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *J. Comput. System Sci.* 65(4):695–716, 2002.

[7] W. Hoeffding. Probability inequalities for sums of bounded random variables. *J. Amer. Statist. Assoc.* 58(301):13–30, 1963.

[8] J. Hopcroft, W. Paul, and L. Valiant. On time versus space and related problems. In *Proc. 16th IEEE Symposium on Foundations of Computer Science*, pages 57–64, 1975.

[9] J. Hopcroft, W. Paul, and L. Valiant. On time versus space. *J. ACM* 24(2):332–337, 1977.

[10] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *J. Comput. System Sci.* 63(4):512–530, 2001.

[11] S. Kintali. A simplified proof of ETH is false. Manuscript, October 26, 2026.

[12] S.-Y. Kuroda. Classes of languages and linear-bounded automata. *Information and Control* 7(2):207–223, 1964.

[13] OpenAI. The Exponential Time Hypothesis is false. Manuscript, 17 October 2026; revised 25 October 2026. https://github.com/openai/math/blob/main/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf.

[14] W. J. Paul, N. Pippenger, E. Szemerédi, and W. T. Trotter. On determinism versus non-determinism and related problems. In *Proc. 24th IEEE FOCS*, pages 429–438, 1983.

[15] E. Pyne and R. Tell. Composing low-space algorithms. ECCC TR25-140, 2025.

[16] R. Santhanam. On separators, segregators and time versus space. In *Proc. 16th IEEE Conference on Computational Complexity*, 2001. ECCC TR01-022.

[17] R. E. Stearns, J. Hartmanis, and P. M. Lewis II. Hierarchies of memory limited computations. In *Proc. 6th Annual Symposium on Switching Circuit Theory and Logical Design*, pages 179–190, 1965.

[18] R. Williams. Simulating time with square-root space. In *Proc. 57th ACM STOC*, 2025. arXiv:2502.17779.